

ESP32 Firmware Style Guide

STAR Development Team

December 15, 2025

Contents

1.0	ASM and Inline ASM	8
1.1	General Guidelines for ASM and Inline ASM	8
1.2	When to Use ASM and Inline ASM	9
2.0	Assertions	9
2.1	Guidelines	9
2.2	ESP32-Specific Assertions	10
3.0	Braces	10
3.1	General Rules	10
3.2	Control Structures	10
3.2.1	For Loops	11
3.2.2	While Loops	12
3.3	Functions	12
3.4	Structs and Enums	13
3.5	Initializer Lists	14
3.6	Closing Brace Placement	15
3.7	Summary Table	16
4.0	Switch Statements	16
4.1	General Rules	17
4.2	Basic Switch Structure	17
4.3	Case Block Braces	18
4.4	Break Statement Requirements	19
4.5	Default Case Handling	21
4.6	Switch on Enum Types	22

4.7	Indentation	23
4.8	Summary	23
5.0	Breaking Long Lines	24
6.0	Comments	24
6.1	General Commenting Guidelines	24
6.2	File Header Comments	24
6.3	Source File Header	26
6.4	Doxygen Documentation	26
6.4.1	Function Documentation	26
6.4.2	Parameter Direction Markers	27
6.4.3	Code Examples with @code/@endcode	28
6.4.4	Struct Member Documentation	29
6.5	Special Comment Keywords	29
6.5.1	Keyword Definitions	30
6.6	Section Comments	30
6.7	Aligned Comments	31
6.8	Implementation Comments	31
6.9	Private Function Documentation	32
6.10	Summary	33
7.0	Concurrency and Parallelism	33
7.1	RTOS Types, Mutexes, Semaphores, and Atomic Operations	33
8.0	Enums	34
8.1	General Rules	34
8.2	Basic Enum Structure	34
8.3	Explicit Value Assignment	35
8.4	Documentation	35
8.5	Enum to String Functions	36
8.6	Using the Count Sentinel	37
8.7	Switch Statement Pattern	37
8.8	Forward Declaration	38
8.9	Naming Convention Summary	39
9.0	Error Handling	39
9.1	General Principles	39
9.2	ESP-IDF Error Codes	39
9.3	ESP-IDF Validation Macros	40

9.3.1	ESP_RETURN_ON_FALSE	40
9.3.2	ESP_GOTO_ON_ERROR	41
9.3.3	ESP_ERROR_CHECK	42
9.4	NULL Safety Patterns	43
9.5	Mutex Error Handling	44
9.6	Error Interface Dependency Injection	45
9.7	Cleanup Patterns	46
9.8	Error Logging	47
9.9	Summary	47
10.0	Functions	48
10.1	Return Value Conventions	48
10.2	Private and Exposed Private Functions	48
10.2.1	Private Functions (static)	48
10.2.2	Exposed Private Functions (priv_)	49
10.3	Parameter Validation	49
10.4	Function Declaration Formatting	51
10.5	Doxygen Documentation	52
10.6	Private Function Documentation	52
10.7	Function Implementation Patterns	53
10.7.1	Single Exit Point with Cleanup	53
10.7.2	Factory Function Pattern	54
10.8	Callback Functions	55
10.9	Summary	56
11.0	Global Variables	56
11.1	General Principles	57
11.2	Static File-Scoped Variables (s_ prefix)	57
11.3	Why static const Over #define	57
11.4	When to Use #define	58
11.5	True Global Variables (g_ prefix)	59
11.6	Dependency Injection Alternative	59
11.7	Static Variables for Module State	60
11.8	Thread Safety for Shared State	61
11.9	Summary	62
12.0	Header Files	62
12.1	General Principles	62

12.2	Include Guard Naming Convention	62
12.3	C++ Compatibility	63
12.4	Header File Structure	64
12.5	Include Order	65
12.6	Forward Declarations	66
12.7	Separating Types from Functions	67
12.8	Doxygen File Documentation	68
12.9	Complete Header Template	68
12.10	Summary	70
13.0	Horizontal Space	70
13.1	General Rules	70
13.2	Space After Control Keywords	71
13.3	No Space After Function Names	72
13.4	Binary Operators	72
13.5	Unary Operators	73
13.6	Pointer Declarations	73
13.7	Comma and Semicolon Spacing	74
13.8	Parentheses Spacing	74
13.9	Parameter Alignment	75
13.10	Variable and Comment Alignment	75
13.11	Struct Initializer Alignment	76
13.12	Summary	77
14.0	Include Statements	77
14.1	General Rules	77
14.2	Include Order	77
14.3	Complete Example from Codebase	78
14.4	FreeRTOS Include Order	79
14.5	System Includes	79
14.6	ESP-IDF Includes	79
14.7	Project Includes	80
14.8	Header File Includes	80
14.9	Conditional Includes	81
14.10	Include Comments	82
14.11	Avoiding Circular Dependencies	82
14.12	Summary	83

15.0	Indentation	83
16.0	Line Endings	83
17.0	Logging	83
18.0	Naming Conventions	83
18.1	General Rules	84
18.2	Functions and Variables	84
18.3	Module Prefixes	84
18.4	Constants and Macros	85
18.5	Type Names	86
18.6	Enum Members	86
18.7	Static Variables (File-Scoped)	87
18.8	Global Variables	88
18.9	Private Functions	88
18.9.1	File-Scoped Static Functions (internal_ prefix)	88
18.9.2	Exposed Private Functions (priv_ prefix)	89
18.10	Summary Table	90
19.0	Type Definitions	90
19.1	Typedef Struct Pattern	90
19.1.1	Standard Pattern	91
19.1.2	Error Handler Example	91
19.2	Union Usage for Protocol Configurations	92
19.2.1	Protocol-Specific Structures	93
19.3	Callback Function Types	93
19.4	Interface Patterns (Dependency Injection)	95
19.4.1	Interface Usage Pattern	96
19.5	Opaque Handle Patterns	97
19.6	Operations Structure Pattern	97
19.7	Manager Structure Pattern	98
19.8	Best Practices	99
20.0	File Organization	99
20.1	Library Directory Structure	99
20.1.1	Example: star_bus Library	99
20.2	CMakeLists.txt Structure	100
20.3	Header File Organization	101
20.3.1	Header File Template	101

20.3.2	Header Section Order	103
20.4	Include Guard Naming Convention	103
20.5	Source File Organization	104
20.5.1	Source File Template	104
20.5.2	Source Section Order	106
20.6	Types Header Separation	106
20.6.1	Controller Header Pattern	106
20.7	Project Root Structure	107
20.8	Best Practices	108
21.0	Inclusive Terminology	108
21.1	Overview	108
21.2	SPI Bus Terminology	108
21.2.1	COPI/CIPO (Not MOSI/MISO)	108
21.3	I2C and General Bus Terminology	110
21.3.1	Controller/Peripheral (Not Master/Slave)	110
21.4	Mapping ESP-IDF Legacy APIs	111
21.5	Documentation Guidelines	112
21.6	Code Review Checklist	113
21.7	Common Patterns	113
21.8	Summary	114
22.0	Architecture Patterns	114
22.1	Dependency Injection (DIP)	114
22.1.1	Interface Definition	114
22.1.2	Interface Implementation	115
22.1.3	Dependency Injection Pattern	115
22.1.4	NULL Interface Handling	116
22.2	Bus Abstraction Pattern	117
22.2.1	Unified Configuration	117
22.2.2	Factory Functions	118
22.2.3	Manager Pattern	118
22.3	Operations Structure Pattern	119
22.4	Thread-Safe Callback Pattern	120
22.5	Thread Safety Patterns	121
22.5.1	Mutex Timeout Convention	121
22.5.2	Mutex with Cleanup (goto pattern)	121
22.6	Error Handler Ownership Pattern	122

22.7	Linked List Management	124
22.8	Best Practices Summary	125

1.0 ASM and Inline ASM

In this project, **Assembly code** and **Inline Assembly** should be used with extreme caution. While it can offer performance improvements or enable access to low-level hardware features, its use can make code harder to understand, maintain, and port. Always prefer writing in C unless absolutely necessary.

1.1 General Guidelines for ASM and Inline ASM

- **Avoid ASM When Possible:** Always try to write in C before resorting to assembly language. C compilers are highly optimized and can often produce efficient machine code that rivals hand-written assembly.
- **Document ASM Code Thoroughly:** Assembly code is often harder to understand than C, so it should be accompanied by thorough comments that explain **why** ASM was used, as well as what the code does.

```
1 /* Set register to zero using ASM */
2 mov r0, #0      /* Initialize r0 to 0 */
```

- **Use Inline ASM Sparingly:** Inline assembly allows you to embed assembly instructions directly in C code. It should only be used when absolutely necessary, such as when interacting with hardware registers or performing CPU-specific optimizations.
- **Constrain the Scope of Inline ASM:** When using inline assembly, constrain its scope to as few lines as possible and avoid intermixing it with complex C logic. This helps isolate potential issues and reduces the risk of bugs.

```
1 void set_flag(void)
2 {
3     asm volatile("mov r0, #1" : : : "r0"); /* Set flag in register r0 */
4 }
```

- **Always Use volatile for Inline ASM:** Inline assembly should be marked as volatile to prevent the compiler from optimizing it out.
- **Do Not Use Inline ASM for Optimizations:** Modern C compilers perform aggressive optimizations. Inline ASM should not be used to try and optimize code performance unless absolutely necessary, as it is often more error-prone than compiler optimizations.
- **Use Constraints Correctly:** When using inline ASM in C, use the appropriate constraints to inform the compiler how the assembly instructions interact with C variables.

```

1 int add(int a, int b)
2 {
3     int result;
4     asm volatile(
5         "add %[r], %[a], %[b]"
6         : [r] "=r" (result)    /* Output */
7         : [a] "r" (a), [b] "r" (b) /* Inputs */
8     );
9     return result;
10 }

```

1.2 When to Use ASM and Inline ASM

- **Hardware Access:** Use ASM when direct hardware access is required and C alone cannot provide the necessary control (e.g., setting or clearing specific registers).
- **Performance Critical Code:** ASM may be justified in performance-critical sections where C compiler optimizations fall short, but this should be documented and used as a last resort.
- **CPU-Specific Instructions:** Use ASM for executing instructions that are unique to the target CPU (e.g., specific ARM or x86 instructions that are not exposed in C).

2.0 Assertions

In this project, **assertions** are used as a tool to catch programming errors and invalid states during development. They help ensure that assumptions made in the code are correct, and they provide valuable debugging information when things go wrong. However, assertions should never be relied on to handle runtime errors or expected conditions in production code.

2.1 Guidelines

- **Use Assertions for Debugging:** Assertions are intended to catch conditions that should never occur during normal program execution. Use them to assert assumptions about the state of the program, such as parameter validation or invariants within algorithms.

```

1 #include <assert.h>
2
3 void process_data(int* data)
4 {

```

```

5  assert(data != NULL); /* Assert that data is not NULL */
6  /* Continue processing */
7  }

```

2.2 ESP32-Specific Assertions

In ESP32 projects, you have two specific macros for error checking: `ESP_ERROR_CHECK` and `ESP_ERROR_CHECK_WITHOUT_ABORT`. These are commonly used to catch errors during development, especially when working with ESP-IDF functions.

- **ESP_ERROR_CHECK:** This macro checks the return value of an ESP-IDF function and aborts the program if the return code is not `ESP_OK`.
- **ESP_ERROR_CHECK_WITHOUT_ABORT:** This macro works similarly to `ESP_ERROR_CHECK`, but it will not abort the program when an error occurs.

3.0 Braces

Proper brace placement makes code cleaner and easier to read. This section establishes consistent brace conventions used throughout the STAR firmware codebase.

3.1 General Rules

- **Always Use Braces for Control Statements:** Every `if`, `for`, `while`, or similar control structure must have braces, even for single statements.
- **Same Line for Control Structures and Blocks:** For control structures like `if`, `for`, `while`, and blocks like structs and enums, always place the opening brace `{` on the same line as the statement.
- **Functions Have Braces on a New Line:** The only exception to the same-line rule is for functions, where the opening brace goes on its own line.

3.2 Control Structures

Opening brace on the **same line** as the control statement.

```

1  /* CORRECT - if/else with braces on same line */
2  if (ret != ESP_OK) {
3      ESP_LOGE(s_TAG, "Operation failed: %s", esp_err_to_name(ret));
4      return ret;

```

```

5  }
6
7  if (manager == NULL) {
8      return ESP_ERR_INVALID_ARG;
9  } else if (manager->mutex == NULL) {
10     return ESP_ERR_INVALID_STATE;
11 } else {
12     /* Proceed with operation */
13 }
14
15 /* CORRECT - Single statement still requires braces */
16 if (pin < 0 || pin >= GPIO_NUM_MAX) {
17     return ESP_OK; /* Not an error, pin is not used */
18 }
19
20 /* WRONG - Missing braces */
21 if (ret != ESP_OK)
22     return ret;
23
24 /* WRONG - Brace on new line */
25 if (ret != ESP_OK)
26 {
27     return ret;
28 }

```

3.2.1 For Loops

```

1  /* CORRECT - for loop with brace on same line */
2  for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
3      if (curr->name && strcmp(curr->name, name) == 0) {
4          found_bus = curr;
5          break;
6      }
7  }
8
9  for (uint32_t i = 0; i < SPI_HOST_MAX; ++i) {
10     manager->spi_host_initialized[i] = false;
11     manager->spi_device_count[i] = 0;

```

```

12 }
13
14 /* WRONG */
15 for (uint32_t i = 0; i < count; ++i)
16     process_item(i); /* Missing braces */

```

3.2.2 While Loops

```

1  /* CORRECT - while loop with brace on same line */
2  while (curr) {
3      star_bus_config_t* next = curr->next;
4      esp_err_t destroy_result = star_bus_config_destroy(curr);
5      if (destroy_result != ESP_OK && first_error == ESP_OK) {
6          first_error = destroy_result;
7      }
8      curr = next;
9  }
10
11 while (error_handler_can_retry(&handler)) {
12     ret = attempt_operation();
13     if (ret == ESP_OK) {
14         break;
15     }
16 }
17
18 /* WRONG */
19 while (curr)
20 {
21     process_node(curr); /* Brace on new line */
22     curr = curr->next;
23 }

```

3.3 Functions

Opening brace on a **new line** for function definitions.

```

1  /* CORRECT - Function brace on new line */
2  esp_err_t star_bus_manager_init(star_bus_manager_t*    manager,

```

```

3         const char*          tag,
4         star_error_interface_t* error_iface,
5         star_pin_interface_t*  pin_iface)
6 {
7     ESP_RETURN_ON_FALSE(manager, ESP_ERR_INVALID_ARG, s_TAG,
8         "Manager pointer is NULL");
9
10    manager->buses      = NULL;
11    manager->error_iface = error_iface;
12    manager->pin_iface  = pin_iface;
13
14    /* ... */
15
16    return ESP_OK;
17 }
18
19 static esp_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
20                                           gpio_num_t          pin,
21                                           const char*          bus_name,
22                                           const char*          usage,
23                                           bool                  can_be_shared)
24 {
25     if (pin < 0 || pin >= GPIO_NUM_MAX) {
26         return ESP_OK;
27     }
28     /* ... */
29 }
30
31 /* WRONG - Function brace on same line */
32 esp_err_t star_bus_manager_init(star_bus_manager_t* manager, ...) {
33     /* ... */
34 }

```

3.4 Structs and Enums

Opening brace on the **same line** as the typedef/struct/enum keyword.

```

1 /* CORRECT - Struct brace on same line */
2 typedef struct star_bus_manager {

```

```

3     star_bus_config_t*      buses;
4     SemaphoreHandle_t      mutex;
5     const char*            tag;
6     star_error_interface_t* error_iface;
7     star_pin_interface_t*   pin_iface;
8     bool                   spi_host_initialized[SPI_HOST_MAX];
9     int                    spi_device_count[SPI_HOST_MAX];
10 } star_bus_manager_t;
11
12 /* CORRECT - Enum brace on same line */
13 typedef enum {
14     k_star_bus_type_none,
15     k_star_bus_type_i2c,
16     k_star_bus_type_spi,
17     k_star_bus_type_gpio,
18     k_star_bus_type_adc,
19     k_star_bus_type_count,
20 } star_bus_type_t;
21
22 /* CORRECT - Union brace on same line */
23 union {
24     star_i2c_bus_config_t i2c;
25     star_spi_bus_config_t spi;
26     star_gpio_bus_config_t gpio;
27     star_adc_bus_config_t adc;
28 } proto;
29
30 /* WRONG - Brace on new line */
31 typedef struct star_bus_manager
32 {
33     /* ... */
34 } star_bus_manager_t;

```

3.5 Initializer Lists

Opening brace on the **same line** as the assignment.

```

1 /* CORRECT - Initializer brace on same line */
2 star_bus_event_t event = {

```

```

3     .bus_type = k_star_bus_type_i2c,
4     .bus_name = config->name,
5     .i2c = {
6         .port = port,
7         .address = address,
8         .is_write = true,
9         .len = len
10    }
11 };
12
13 spi_device_interface_config_t spi_dev_cfg = {
14     .clock_speed_hz = 10 * 1000 * 1000,
15     .mode = 0,
16     .spics_io_num = GPIO_NUM_5,
17     .queue_size = 7,
18     .pre_cb = NULL,
19     .post_cb = NULL,
20 };
21
22 /* CORRECT - Short initializers can be on one line */
23 i2c_config_t i2c_conf = {.mode = I2C_MODE_MASTER};
24
25 /* WRONG - Brace on new line */
26 star_bus_event_t event =
27 {
28     .bus_type = k_star_bus_type_i2c,
29     /* ... */
30 };

```

3.6 Closing Brace Placement

Closing braces are always on their own line, except for else, else if, and while in do-while.

```

1 /* CORRECT - else on same line as closing brace */
2 if (ret != ESP_OK) {
3     handle_error(ret);
4 } else {
5     continue_operation();
6 }

```

```

7
8  /* CORRECT - do-while */
9  do {
10     ret = try_operation();
11 } while (ret == ESP_ERR_TIMEOUT && retries-- > 0);
12
13 /* CORRECT - else if chain */
14 if (config->type == k_star_bus_type_i2c) {
15     /* Handle I2C */
16 } else if (config->type == k_star_bus_type_spi) {
17     /* Handle SPI */
18 } else {
19     /* Handle other */
20 }

```

3.7 Summary Table

Construct	Opening Brace Position
Functions	New line
if/else/else if	Same line
for	Same line
while	Same line
do-while	Same line
switch	Same line
struct	Same line
enum	Same line
union	Same line
Initializer lists	Same line

4.0 Switch Statements

Switch statements follow specific formatting rules to ensure consistency and readability throughout the STAR firmware codebase.

4.1 General Rules

- **Same Line for Switch and Opening Brace:** The opening brace { for a switch statement should be on the same line as the switch keyword.
- **Braces for Case Blocks:** Each case should have its own block of code enclosed in braces {}.
- **Break Statements:** Every case must end with break, return, or a fall-through comment.
- **Default Case:** Always include a default case, even if it only logs a warning.

4.2 Basic Switch Structure

```
1  /* CORRECT - from star_bus_manager.c */
2  switch (config->type) {
3      case k_star_bus_type_i2c: {
4          ret = internal_register_bus_pin(pin_iface,
5                                          config->proto.i2c.config.sda_io_num,
6                                          config->name,
7                                          "I2C SDA",
8                                          true);
9          if (ret != ESP_OK && first_error == ESP_OK) {
10             first_error = ret;
11         }
12
13         ret = internal_register_bus_pin(pin_iface,
14                                         config->proto.i2c.config.scl_io_num,
15                                         config->name,
16                                         "I2C SCL",
17                                         true);
18         if (ret != ESP_OK && first_error == ESP_OK) {
19             first_error = ret;
20         }
21         break;
22     }
23
24     case k_star_bus_type_spi: {
25         ret = internal_register_bus_pin(pin_iface,
26                                         config->proto.spi.copi_io_num,
27                                         config->name,
```

```

28         "SPI COPI",
29         true);
30     if (ret != ESP_OK && first_error == ESP_OK) {
31         first_error = ret;
32     }
33     /* ... more pin registrations ... */
34     break;
35 }
36
37 default: {
38     ESP_LOGD(s_TAG, "No pin registration for bus type: %d", config->type);
39     break;
40 }
41 }

```

4.3 Case Block Braces

Each case must have its own braces. This prevents variable scope issues and improves readability.

```

1  /* CORRECT - Each case has braces */
2  switch (bus_type) {
3      case k_star_bus_type_i2c: {
4          i2c_port_t port = config->proto.i2c.port; /* Local variable */
5          uint8_t address = config->proto.i2c.address;
6          ESP_LOGI(s_TAG, "I2C bus on port %d, address 0x%02X", port, address);
7          break;
8      }
9      case k_star_bus_type_spi: {
10         spi_host_device_t host = config->proto.spi.host; /* Different scope */
11         ESP_LOGI(s_TAG, "SPI bus on host %d", host);
12         break;
13     }
14     case k_star_bus_type_gpio: {
15         uint8_t pin_count = config->proto.gpio.pin_count;
16         ESP_LOGI(s_TAG, "GPIO bus with %d pins", pin_count);
17         break;
18     }
19     default: {
20         ESP_LOGW(s_TAG, "Unknown bus type: %d", bus_type);

```

```

21         break;
22     }
23 }
24
25 /* WRONG - Missing braces */
26 switch (bus_type) {
27     case k_star_bus_type_i2c:
28         handle_i2c();
29         break;
30     case k_star_bus_type_spi:
31         handle_spi();
32         break;
33 }

```

4.4 Break Statement Requirements

Every case must have a clear exit: break, return, or explicit fall-through.

```

1  /* CORRECT - Using break */
2  switch (config->type) {
3      case k_star_bus_type_i2c: {
4          process_i2c(config);
5          break;
6      }
7      case k_star_bus_type_spi: {
8          process_spi(config);
9          break;
10     }
11     default: {
12         break;
13     }
14 }
15
16 /* CORRECT - Using return */
17 const char* star_bus_type_to_string(star_bus_type_t type)
18 {
19     switch (type) {
20         case k_star_bus_type_none: {
21             return "NONE";

```

```

22     }
23     case k_star_bus_type_i2c: {
24         return "I2C";
25     }
26     case k_star_bus_type_spi: {
27         return "SPI";
28     }
29     case k_star_bus_type_gpio: {
30         return "GPIO";
31     }
32     case k_star_bus_type_adc: {
33         return "ADC";
34     }
35     case k_star_bus_type_count: {
36         return "COUNT";
37     }
38     default: {
39         return "UNKNOWN";
40     }
41 }
42 }
43
44 /* CORRECT - Explicit fall-through (rare, must be commented) */
45 switch (priority) {
46     case PRIORITY_CRITICAL: {
47         log_critical(message);
48         /* FALLTHROUGH */
49     }
50     case PRIORITY_HIGH: {
51         notify_admin(message);
52         /* FALLTHROUGH */
53     }
54     case PRIORITY_NORMAL: {
55         log_message(message);
56         break;
57     }
58     default: {
59         break;

```

```

60     }
61 }

```

4.5 Default Case Handling

Always include a default case. Use it for error handling or logging unexpected values.

```

1  /* CORRECT - Default case logs warning */
2  switch (curr->type) {
3      case k_star_bus_type_i2c: {
4          reg_result = register_pin_if_valid(curr->proto.i2c.config.sda_io_num,
5                                          bus_name, "I2C SDA", reg_func,
6                                          user_ctx, true);
7
8          /* ... */
9          break;
10     }
11     case k_star_bus_type_spi: {
12         reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
13                                         bus_name, "SPI COPI", reg_func,
14                                         user_ctx, true);
15
16         /* ... */
17         break;
18     }
19     default: {
20         ESP_LOGW(manager->tag, "Skipping pin registration for "
21                     "unsupported bus type: %d", curr->type);
22         break;
23     }
24 }
25
26 /* CORRECT - Default case returns error for invalid input */
27 esp_err_t process_bus_type(star_bus_type_t type)
28 {
29     switch (type) {
30         case k_star_bus_type_i2c: {
31             return process_i2c();
32         }
33         case k_star_bus_type_spi: {
34             return process_spi();
35         }
36     }
37 }

```

```

33     }
34     default: {
35         ESP_LOGE(s_TAG, "Invalid bus type: %d", type);
36         return ESP_ERR_INVALID_ARG;
37     }
38 }
39 }

```

4.6 Switch on Enum Types

When switching on an enum, consider handling all values explicitly.

```

1  /* CORRECT - All enum values handled */
2  switch (config->type) {
3      case k_star_bus_type_none: {
4          ESP_LOGW(s_TAG, "Bus type is NONE - skipping");
5          break;
6      }
7      case k_star_bus_type_i2c: {
8          /* Initialize I2C */
9          break;
10     }
11     case k_star_bus_type_spi: {
12         /* Initialize SPI */
13         break;
14     }
15     case k_star_bus_type_gpio: {
16         /* Initialize GPIO */
17         break;
18     }
19     case k_star_bus_type_adc: {
20         /* Initialize ADC */
21         break;
22     }
23     case k_star_bus_type_count: {
24         /* Sentinel value - should never be used */
25         ESP_LOGE(s_TAG, "Invalid bus type: count sentinel");
26         return ESP_ERR_INVALID_ARG;
27     }

```

```

28     /* No default needed when all enum values are handled */
29     /* Compiler warns if new enum values are added */
30 }

```

4.7 Indentation

Case labels are at the same indentation level as the switch. Case body is indented one level.

```

1  /* CORRECT indentation */
2  switch (state) {
3      case STATE_IDLE: {
4          /* Case body indented */
5          if (condition) {
6              /* Nested code further indented */
7          }
8          break;
9      }
10     case STATE_RUNNING: {
11         process_running();
12         break;
13     }
14     default: {
15         break;
16     }
17 }
18
19 /* WRONG - Case labels indented */
20 switch (state) {
21     case STATE_IDLE: {
22         break;
23     }
24 }

```

4.8 Summary

- Opening brace on same line as switch
- Each case has its own braces {...}
- Every case ends with break, return, or fall-through comment

- Always include default case
- Case labels at switch indentation level
- Case body indented one level inside braces

5.0 Breaking Long Lines

The maximum line length is **80 characters**. Lines longer than this should be split.

- **Break at Natural Points:** Break lines at natural points, such as after operators.
- **Indentation for Continuation Lines:** When breaking lines, ensure the continued line is indented properly.

6.0 Comments

All functions must include Doxygen comments. This section establishes comprehensive commenting standards for the STAR firmware codebase.

6.1 General Commenting Guidelines

- **Use Block Comments** (`/* */`): Always prefer block comments over line comments (`//`).
- **Explain Why, Not What:** Comments should explain the reasoning behind code, not what the code does (which should be clear from the code itself).
- **Keep Comments Updated:** Outdated comments are worse than no comments.
- **English Only:** All comments must be in English.

```

1  /* CORRECT - Block comment style */
2  /* Initialize the mutex for thread-safe access */
3  manager->mutex = xSemaphoreCreateMutex();
4
5  /* WRONG - Line comment style */
6  // Initialize the mutex for thread-safe access
7  manager->mutex = xSemaphoreCreateMutex();

```

6.2 File Header Comments

Every source and header file should begin with a file path comment, followed by include guards (for headers) and a `@file` Doxygen block for headers with significant content.

```

1  /* lib/star_bus/include/star_bus_manager.h */
2
3  #ifndef STAR_COMPONENT_BUS_MANAGER_H
4  #define STAR_COMPONENT_BUS_MANAGER_H
5
6  #ifdef __cplusplus
7  extern "C" {
8  #endif
9
10 #include "esp_err.h"
11 #include "star_bus_manager_types.h"
12
13 /**
14  * @file star_bus_manager.h
15  * @brief Unified bus manager for I2C, SPI, GPIO, and other bus protocols
16  *
17  * The bus manager provides a central registry for managing multiple bus
18  * configurations. It supports dependency injection of error handlers and
19  * pin validators for loose coupling.
20  *
21  * Key Features:
22  * - Thread-safe bus management with mutex protection
23  * - Support for multiple bus types (I2C, SPI, GPIO, DHT22, etc.)
24  * - Automatic pin registration and conflict detection
25  * - Safe bus access via callback pattern (prevents use-after-free)
26  *
27  * Example Usage:
28  * @code
29  * // === Basic Bus Manager Setup ===
30  *
31  * #include "star_bus_manager.h"
32  * #include "star_bus_config.h"
33  * #include "star_error_handler.h"
34  *
35  * // 1. Create error handler and pin validator interfaces
36  * error_handler_t error_handler;
37  * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
38  *

```

```

39 * star_error_interface_t error_iface;
40 * star_pin_interface_t pin_iface;
41 * error_handler_get_interface(&error_iface, &error_handler);
42 * pin_validator_get_interface(&pin_iface);
43 *
44 * // 2. Initialize bus manager with dependency injection
45 * star_bus_manager_t bus_manager;
46 * star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
47 * @endcode
48 */

```

6.3 Source File Header

Source files begin with a file path comment.

```

1 /* lib/star_bus/src/star_bus_manager.c */
2
3 #include "star_bus_manager.h"
4
5 #include "freertos/FreeRTOS.h"
6 #include "freertos/semphr.h"
7
8 /* ... */

```

6.4 Doxygen Documentation

6.4.1 Function Documentation

All public functions must have Doxygen documentation in the header file.

```

1 /**
2  * @brief Initialize a bus manager instance.
3  *
4  * Allocates resources (like a mutex) for the manager. Must be called
5  * before any other manager functions.
6  *
7  * @param[out] manager      Pointer to bus manager structure to initialize.
8  *                          Must not be NULL.
9  * @param[in]  tag          Optional tag string for logging associated with
10 *                          this manager instance. If NULL or empty, a

```

```

11      *                      default tag is used.
12      * @param[in]  error_iface Optional error handler interface (can be NULL).
13      * @param[in]  pin_iface   Optional pin validator interface (can be NULL).
14      * @return esp_err_t ESP_OK on success, ESP_ERR_INVALID_ARG if manager
15      *                      is NULL, ESP_ERR_NO_MEM if mutex creation fails.
16      */
17      esp_err_t star_bus_manager_init(star_bus_manager_t*    manager,
18                                     const char*            tag,
19                                     star_error_interface_t* error_iface,
20                                     star_pin_interface_t*   pin_iface);

```

6.4.2 Parameter Direction Markers

Use [in], [out], or [in,out] to indicate parameter direction.

```

1  /**
2   * @brief Write data to an I2C device register.
3   *
4   * @param[in]  config      Pointer to I2C bus configuration.
5   * @param[in]  data        Pointer to data buffer to write.
6   * @param[in]  len         Number of bytes to write.
7   * @param[in]  reg_addr    Target register address.
8   * @param[out] bytes_written Pointer to store actual bytes written.
9   *
10      *                      Can be NULL if not needed.
11      * @return esp_err_t ESP_OK on success.
12      */
13      static esp_err_t internal_star_bus_i2c_write(const star_bus_config_t* config,
14                                                    const uint8_t*      data,
15                                                    size_t          len,
16                                                    uint8_t          reg_addr,
17                                                    size_t*
18      bytes_written);
19  /**
20   * @brief Record an error and manage retry/reset logic
21   *
22   * @param[in,out] handler  Pointer to error handler structure
23   * @param[in]     error     Error code that occurred
24   * @param[in]     description Human-readable description of the error

```

```

24 * @param[in]    file        Source file where error occurred
25 * @param[in]    line        Line number where error occurred
26 * @param[in]    func        Function name where error occurred
27 * @return ESP_OK if error was handled, or the original error code.
28 */
29 esp_err_t error_handler_record_error(error_handler_t* handler,
30                                     esp_err_t      error,
31                                     const char*      description,
32                                     const char*      file,
33                                     int32_t          line,
34                                     const char*      func);

```

6.4.3 Code Examples with @code/@endcode

Include usage examples in documentation blocks.

```

1 /**
2  * @file star_error_interface.h
3  * @brief Error handler interface for dependency inversion
4  *
5  * Example Usage:
6  * @code
7  * // === Creating and Using Error Interface ===
8  *
9  * #include "star_error_interface.h"
10 * #include "star_error_handler.h"
11 *
12 * // 1. Create concrete error handler
13 * error_handler_t error_handler;
14 * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
15 *
16 * // 2. Get interface from implementation
17 * star_error_interface_t error_iface;
18 * error_handler_get_interface(&error_iface, &error_handler);
19 *
20 * // 3. Inject into components that need error handling
21 * star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
22 *
23 * // 4. Use the interface

```

```

24 * if (error_iface.can_retry(error_iface.ctx)) {
25 *     // Retry operation...
26 * }
27 * @endcode
28 */

```

6.4.4 Struct Member Documentation

Document struct members inline with Doxygen.

```

1 /**
2  * @brief Bus configuration structure (common part).
3  */
4 typedef struct star_bus_config {
5     const char*    name;        /**< Unique name for this bus/device instance */
6     star_bus_type_t type;        /**< Type of bus (I2C, SPI, etc.) */
7     bool           initialized; /**< Whether this bus has been initialized */
8     void*          handle;       /**< Driver/device handle */
9     void*          user_ctx;     /**< User context pointer for callbacks */
10
11     union {
12         star_i2c_bus_config_t i2c; /**< I2C-specific configuration */
13         star_spi_bus_config_t spi; /**< SPI-specific configuration */
14         star_gpio_bus_config_t gpio; /**< GPIO-specific configuration */
15         star_adc_bus_config_t adc; /**< ADC-specific configuration */
16     } proto;
17
18     struct star_bus_config* next; /**< Next bus config in linked list */
19 } star_bus_config_t;

```

6.5 Special Comment Keywords

Use these keywords to mark special comments for tracking.

```

1 /* TODO: Implement retry logic for I2C bus reset */
2
3 /* FIXME: Memory leak when bus removal fails mid-operation */
4
5 /* BUG: Race condition possible if mutex timeout is too short */

```

```

6
7 /* XXX: This code assumes little-endian byte order */
8
9 /* NOTE: ESP-IDF uses mosi_io_num internally - we map to COPI here */
10
11 /* WARNING: Must hold mutex before calling this function */

```

6.5.1 Keyword Definitions

- **TODO:** Work that needs to be done but is not urgent.
- **FIXME:** Known bugs or issues that need to be fixed.
- **BUG:** Confirmed bugs in the code.
- **XXX:** Dangerous or hacky code that works but should be reviewed.
- **NOTE:** Important information about the code.
- **WARNING:** Critical information that must be understood before modifying.

6.6 Section Comments

Use section comments to organize code within files.

```

1 /* === Includes === */
2 #include "star_bus_manager.h"
3 #include "freertos/FreeRTOS.h"
4
5 /* === Constants === */
6 static const char* s_TAG = "STAR_BUS_MANAGER";
7
8 /* === Private Function Prototypes === */
9 static esp_err_t internal_register_bus_pin(...);
10
11 /* === Public Functions === */
12 esp_err_t star_bus_manager_init(...) { /* ... */ }
13
14 /* === Private Functions === */
15 static esp_err_t internal_register_bus_pin(...) { /* ... */ }
16
17 /* --- Pin Registration Helpers --- */
18 /* Subsection within a section */

```

6.7 Aligned Comments

Align comments after declarations for improved readability.

```
1 typedef struct error_handler {
2     uint32_t      error_count;          /**< Total errors recorded */
3     uint32_t      max_retries;          /**< Max retry attempts */
4     uint32_t      current_retry;        /**< Current retry counter */
5     uint32_t      base_retry_delay;     /**< Base delay (ms) */
6     uint32_t      max_retry_delay;      /**< Max delay (ms) */
7     uint32_t      current_retry_delay;  /**< Current delay with backoff */
8     esp_err_t     last_error;           /**< Last recorded error code */
9     bool          in_error_state;        /**< Whether in error state */
10    void*          reset_context;         /**< Context for reset function */
11    SemaphoreHandle_t mutex;              /**< Mutex for thread-safety */
12
13    esp_err_t (*reset_fn)(void* context); /**< Optional reset function */
14 } error_handler_t;
```

6.8 Implementation Comments

Comments within function bodies explain non-obvious logic.

```
1 esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2                                     star_bus_config_t*  config)
3 {
4     /* Validate inputs using ESP-IDF macro */
5     ESP_RETURN_ON_FALSE(manager && config, ESP_ERR_INVALID_ARG,
6                          s_TAG, "Manager or config pointer is NULL");
7
8     esp_err_t ret = ESP_OK;
9
10    /* Acquire mutex with timeout to prevent deadlock */
11    if (xSemaphoreTake(manager->mutex,
12                        pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
13        != pdTRUE) {
14        ESP_LOGE(manager->tag, "Timeout acquiring mutex");
15        return ESP_ERR_TIMEOUT;
16    }
17 }
```

```

18  /* Check for duplicate name - bus names must be unique */
19  for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
20      if (curr->name && strcmp(curr->name, config->name) == 0) {
21          ret = ESP_ERR_INVALID_STATE;
22          goto add_give_mutex; /* Single exit point pattern */
23      }
24  }
25
26  /* Initialize hardware before adding to list
27   * This ensures manager's SPI host tracking is updated correctly */
28  if (!config->initialized) {
29      ret = star_bus_config_init(config, manager);
30      if (ret != ESP_OK) {
31          goto add_give_mutex;
32      }
33  }
34
35  /* Add to head of list (O(1) insertion) */
36  config->next = manager->buses;
37  manager->buses = config;
38
39  add_give_mutex:
40      xSemaphoreGive(manager->mutex);
41      return ret;
42  }

```

6.9 Private Function Documentation

Static functions should have brief documentation in the source file.

```

1  /**
2   * @brief Helper to register a single pin via the interface
3   */
4  static esp_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
5                                             gpio_num_t          pin,
6                                             const char*         bus_name,
7                                             const char*         usage,
8                                             bool                 can_be_shared)
9  {

```

```

10     if (pin < 0 || pin >= GPIO_NUM_MAX) {
11         return ESP_OK; /* Not an error, pin is not used */
12     }
13
14     if (!pin_iface || !pin_iface->register_pin) {
15         return ESP_OK; /* No interface, skip registration */
16     }
17
18     /* Create description: "BusName: Usage" (e.g., "I2C0: SDA") */
19     char desc[64];
20     snprintf(desc, sizeof(desc), "%s: %s", bus_name, usage);
21
22     return pin_iface->register_pin(pin_iface->ctx, pin, desc, can_be_shared);
23 }

```

6.10 Summary

- Use block comments (`/* */`), never line comments (`//`)
- Document all public functions with Doxygen in headers
- Use `[in]`, `[out]`, `[in,out]` for parameter direction
- Include `@code/@endcode` examples in file and complex function docs
- Use `TODO`, `FIXME`, `BUG`, `XXX`, `NOTE`, `WARNING` keywords for tracking
- Align comments after struct members for readability
- Use section comments to organize source files

7.0 Concurrency and Parallelism

This section outlines the best practices for ensuring thread-safe and efficient concurrent programming.

7.1 RTOS Types, Mutexes, Semaphores, and Atomic Operations

- **RTOS Types:** Use RTOS types when developing for embedded systems like ESP32.
- **Mutexes:** Use a mutex when you need **exclusive access** to a shared resource.
- **Semaphores:** Use semaphores when you need to manage **shared resources** that multiple threads can access simultaneously.

- **Atomic Operations:** Use atomic operations when working with **simple data types** such as counters or flags.

8.0 Enums

This section establishes the standards for defining and using enumerations in the STAR firmware codebase.

8.1 General Rules

- **Enum Names Should End in _t:** All enum types should end with `_t`.
- **Use Snake Case for Enum Values:** All enum values should follow `snake_case` naming conventions.
- **Enum Values Should Start with k_:** To distinguish enum members from other constants, values should start with `k_`.
- **Use typedef:** Always use `typedef` to define the enum type.
- **Include Count Sentinel:** Include a `_count` sentinel value for iteration.

8.2 Basic Enum Structure

```

1  /* CORRECT - from star_bus_common_types.h */
2  typedef enum {
3      k_star_bus_type_none,  /**< No bus type specified */
4      k_star_bus_type_i2c,   /**< I2C bus */
5      k_star_bus_type_spi,   /**< SPI bus */
6      k_star_bus_type_gpio,  /**< GPIO bus */
7      k_star_bus_type_adc,   /**< ADC bus */
8      k_star_bus_type_count, /**< Number of bus types (sentinel) */
9  } star_bus_type_t;
10
11 /* WRONG - Various style violations */
12 typedef enum {
13     STAR_BUS_TYPE_I2C,      /* Missing k_ prefix */
14     StarBusTypeSpi,         /* PascalCase not allowed */
15     star_bus_type_gpio,     /* Missing k_ prefix */
16 } bad_enum_style;          /* Missing _t suffix */

```

8.3 Explicit Value Assignment

Assign explicit values when needed for stability or compatibility.

```
1  /* CORRECT - Explicit values for protocol compatibility */
2  typedef enum {
3      k_error_none    = 0,  /**< No error */
4      k_error_minor   = 1,  /**< Minor error, operation can continue */
5      k_error_major    = 2,  /**< Major error, operation should stop */
6      k_error_fatal    = 3,  /**< Fatal error, system needs reset */
7      k_error_count    = 4,  /**< Number of error levels */
8  } error_level_t;
9
10 /* CORRECT - Bit flags with explicit values */
11 typedef enum {
12     k_bus_flag_none      = 0x00,  /**< No flags */
13     k_bus_flag_initialized = 0x01,  /**< Bus is initialized */
14     k_bus_flag_busy      = 0x02,  /**< Bus is busy */
15     k_bus_flag_error      = 0x04,  /**< Bus has error */
16     k_bus_flag_dma        = 0x08,  /**< DMA enabled */
17 } bus_flags_t;
```

8.4 Documentation

Document each enum value with Doxygen comments.

```
1  /**
2   * @brief Types of supported bus interfaces.
3   */
4  typedef enum {
5      k_star_bus_type_none,  /**< No bus type specified */
6      k_star_bus_type_i2c,   /**< I2C bus */
7      k_star_bus_type_spi,   /**< SPI bus */
8      k_star_bus_type_gpio,  /**< GPIO bus */
9      k_star_bus_type_adc,   /**< ADC bus */
10     k_star_bus_type_count,  /**< Number of bus types */
11 } star_bus_type_t;
```

8.5 Enum to String Functions

Provide a string conversion function for each enum type.

```
1  /* In header file */
2  const char* star_bus_type_to_string(star_bus_type_t type);
3
4  /* In source file */
5  const char* star_bus_type_to_string(star_bus_type_t type)
6  {
7      switch (type) {
8          case k_star_bus_type_none: {
9              return "NONE";
10             }
11          case k_star_bus_type_i2c: {
12              return "I2C";
13             }
14          case k_star_bus_type_spi: {
15              return "SPI";
16             }
17          case k_star_bus_type_gpio: {
18              return "GPIO";
19             }
20          case k_star_bus_type_adc: {
21              return "ADC";
22             }
23          case k_star_bus_type_count: {
24              return "COUNT";
25             }
26          default: {
27              return "UNKNOWN";
28             }
29      }
30  }
31
32  /* Usage */
33  ESP_LOGI(s_TAG, "Added bus config: %s (Type: %s)",
34          config->name, star_bus_type_to_string(config->type));
```

8.6 Using the Count Sentinel

The `_count` value enables iteration and array sizing.

```
1  /* Array indexed by enum */
2  static const char* s_bus_type_names[k_star_bus_type_count] = {
3      [k_star_bus_type_none] = "NONE",
4      [k_star_bus_type_i2c]  = "I2C",
5      [k_star_bus_type_spi]  = "SPI",
6      [k_star_bus_type_gpio] = "GPIO",
7      [k_star_bus_type_adc]  = "ADC",
8  };
9
10 /* Iteration using count */
11 for (star_bus_type_t type = k_star_bus_type_none;
12      type < k_star_bus_type_count; ++type) {
13     ESP_LOGI(s_TAG, "Bus type %d: %s", type, s_bus_type_names[type]);
14 }
15
16 /* Validation using count */
17 if (type >= k_star_bus_type_count) {
18     ESP_LOGE(s_TAG, "Invalid bus type: %d", type);
19     return ESP_ERR_INVALID_ARG;
20 }
```

8.7 Switch Statement Pattern

Always handle all enum values or include a default case.

```
1  /* All values handled explicitly */
2  switch (config->type) {
3      case k_star_bus_type_none: {
4          ESP_LOGW(s_TAG, "Bus type is NONE - skipping");
5          break;
6      }
7      case k_star_bus_type_i2c: {
8          ret = init_i2c_bus(config);
9          break;
10     }
11     case k_star_bus_type_spi: {
```

```

12     ret = init_spi_bus(config);
13     break;
14 }
15 case k_star_bus_type_gpio: {
16     ret = init_gpio_bus(config);
17     break;
18 }
19 case k_star_bus_type_adc: {
20     ret = init_adc_bus(config);
21     break;
22 }
23 case k_star_bus_type_count: {
24     /* Sentinel value should never be used */
25     ESP_LOGE(s_TAG, "Invalid bus type: count sentinel");
26     return ESP_ERR_INVALID_ARG;
27 }
28 /* No default - compiler warns if new enum values added */
29 }

```

8.8 Forward Declaration

Enums can be forward-declared for header dependency reduction.

```

1  /* Forward declaration (C11 or later with explicit underlying type) */
2  typedef enum star_bus_type star_bus_type_t;
3
4  /* Or use typedef in header, full definition in source */
5  /* In header: */
6  typedef enum star_bus_type star_bus_type_t;
7
8  /* In source or types header: */
9  enum star_bus_type {
10     k_star_bus_type_none,
11     k_star_bus_type_i2c,
12     /* ... */
13 };

```

8.9 Naming Convention Summary

Element	Convention	Example
Type name	snake_case_t	star_bus_type_t
Member prefix	k_	k_star_bus_type_i2c
Member naming	snake_case	k_star_bus_type_i2c
Count sentinel	_count	k_star_bus_type_count
String function	_to_string	star_bus_type_to_string

9.0 Error Handling

This section establishes error handling patterns for the STAR firmware codebase, with emphasis on ESP-IDF specific patterns.

9.1 General Principles

- **Return Error Codes:** Functions should return error codes via `esp_err_t`.
- **Check All Return Values:** Never ignore return values from functions that can fail.
- **Use goto for Clean-up:** Use `goto` to jump to a clean-up section when multiple resources need release.
- **Fail Fast:** Return early on invalid arguments or precondition failures.

9.2 ESP-IDF Error Codes

Use standard ESP-IDF error codes throughout the codebase.

```
1  /* Standard ESP-IDF error codes */
2  ESP_OK                          /* Success */
3  ESP_FAIL                       /* Generic failure */
4  ESP_ERR_NO_MEM                 /* Out of memory */
5  ESP_ERR_INVALID_ARG           /* Invalid argument */
6  ESP_ERR_INVALID_STATE         /* Invalid state for operation */
7  ESP_ERR_INVALID_SIZE          /* Invalid size */
8  ESP_ERR_NOT_FOUND             /* Requested resource not found */
9  ESP_ERR_NOT_SUPPORTED         /* Operation not supported */
10 ESP_ERR_TIMEOUT               /* Operation timed out */
11
12 /* Usage example */
```

```

13 esp_err_t star_bus_manager_init(star_bus_manager_t* manager, ...)
14 {
15     if (manager == NULL) {
16         return ESP_ERR_INVALID_ARG;
17     }
18
19     manager->mutex = xSemaphoreCreateMutex();
20     if (manager->mutex == NULL) {
21         return ESP_ERR_NO_MEM;
22     }
23
24     return ESP_OK;
25 }

```

9.3 ESP-IDF Validation Macros

Use ESP-IDF macros for parameter validation and error checking.

9.3.1 ESP_RETURN_ON_FALSE

Returns an error if condition is false, with logging.

```

1 #include "esp_check.h"
2
3 esp_err_t star_bus_manager_init(star_bus_manager_t*    manager,
4                                const char*            tag,
5                                star_error_interface_t* error_iface,
6                                star_pin_interface_t*   pin_iface)
7 {
8     /* Validate required parameter with automatic logging */
9     ESP_RETURN_ON_FALSE(manager, ESP_ERR_INVALID_ARG, s_TAG,
10                        "Manager pointer is NULL");
11
12     /* Multiple validations */
13     ESP_RETURN_ON_FALSE(manager->mutex,
14                        ESP_ERR_INVALID_STATE,
15                        manager->tag,
16                        "Manager mutex not initialized");
17

```

```

18     ESP_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
19                          config->type < k_star_bus_type_count,
20                          ESP_ERR_INVALID_ARG,
21                          manager->tag,
22                          "Invalid bus type in config");
23
24     return ESP_OK;
25 }

```

9.3.2 ESP_GOTO_ON_ERROR

Jumps to a label on error, with logging. Use for cleanup patterns.

```

1 static esp_err_t internal_star_bus_i2c_write(const star_bus_config_t* config,
2                                              const uint8_t*      data,
3                                              size_t             len,
4                                              uint8_t           reg_addr,
5                                              size_t*
6 bytes_written)
7 {
8     esp_err_t      ret = ESP_OK;
9     i2c_cmd_handle_t cmd = NULL;
10
11     cmd = i2c_cmd_link_create();
12     ESP_RETURN_ON_FALSE(cmd, ESP_ERR_NO_MEM, s_TAG,
13                         "Failed to create I2C command link");
14
15     ret = i2c_master_start(cmd);
16     ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
17                     "Write '%s': START failed: %s",
18                     config->name, esp_err_to_name(ret));
19
20     ret = i2c_master_write_byte(cmd, (address << 1) | I2C_MASTER_WRITE, true);
21     ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
22                     "Write '%s': Address (W) failed: %s",
23                     config->name, esp_err_to_name(ret));
24
25     ret = i2c_master_write(cmd, data, len, true);
26     ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,

```

```

26         "Write '%s': Data write failed: %s",
27         config->name, esp_err_to_name(ret));
28
29     ret = i2c_master_stop(cmd);
30     ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
31         "Write '%s': STOP failed: %s",
32         config->name, esp_err_to_name(ret));
33
34     /* Execute command */
35     ret = i2c_master_cmd_begin(port, cmd, pdMS_TO_TICKS(1000));
36     ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
37         "Write '%s': CMD failed", config->name);
38
39     /* Success path */
40     i2c_cmd_link_delete(cmd);
41     if (bytes_written) {
42         *bytes_written = len;
43     }
44     return ESP_OK;
45
46 write_fail:
47     if (cmd) {
48         i2c_cmd_link_delete(cmd);
49     }
50     return ret;
51 }

```

9.3.3 ESP_ERROR_CHECK

Use during development to catch fatal errors.

```

1  /* Development only - aborts on error */
2  ESP_ERROR_CHECK(i2c_driver_install(port, I2C_MODE_MASTER, 0, 0, 0));
3
4  /* Use ESP_ERROR_CHECK_WITHOUT_ABORT for non-fatal checks */
5  ESP_ERROR_CHECK_WITHOUT_ABORT(optional_init());

```

9.4 NULL Safety Patterns

Always validate pointer parameters.

```
1  /* At function start - validate all pointers */
2  esp_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
3                               const char*          name,
4                               const uint8_t*        data,
5                               size_t                len,
6                               uint8_t              reg_addr,
7                               size_t*              bytes_written)
8  {
9      ESP_RETURN_ON_FALSE(manager && name && data,
10                          ESP_ERR_INVALID_ARG,
11                          s_TAG,
12                          "Manager, name, or data is NULL");
13      ESP_RETURN_ON_FALSE(len > 0, ESP_ERR_INVALID_ARG, s_TAG,
14                          "Write length must be > 0");
15
16      /* Clear output parameter initially */
17      if (bytes_written) {
18          *bytes_written = 0;
19      }
20
21      /* ... rest of implementation ... */
22  }
23
24  /* Check optional interface before use */
25  if (manager->error_iface && manager->error_iface->record_error) {
26      manager->error_iface->record_error(
27          manager->error_iface->ctx,
28          ret,
29          "Bus hardware initialization failed",
30          __FILE__,
31          __LINE__,
32          __func__);
33  }
34
35  /* Use convenience macro for safe interface calls */
```

```

36 #define STAR_IFACE_RECORD_ERROR(iface, err, msg) \
37     ((iface) && (iface)->record_error \
38         ? (iface)->record_error((iface)->ctx, (err), (msg), \
39             __FILE__, __LINE__, __func__) \
40         : (err))
41
42 /* Usage */
43 STAR_IFACE_RECORD_ERROR(manager->error_iface, ret, "Operation failed");

```

9.5 Mutex Error Handling

Handle mutex acquisition failures properly.

```

1  esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2                                     star_bus_config_t*  config)
3  {
4      /* Validate arguments first */
5      ESP_RETURN_ON_FALSE(manager && config, ESP_ERR_INVALID_ARG,
6                          s_TAG, "Invalid arguments");
7      ESP_RETURN_ON_FALSE(manager->mutex, ESP_ERR_INVALID_STATE,
8                          manager->tag, "Mutex not initialized");
9
10     esp_err_t ret = ESP_OK;
11
12     /* Acquire mutex with timeout */
13     if (xSemaphoreTake(manager->mutex,
14         pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
15         != pdTRUE) {
16         ESP_LOGE(manager->tag, "Timeout acquiring mutex for adding bus '%s'",
17             config->name);
18         return ESP_ERR_TIMEOUT;
19     }
20
21     /* Critical section */
22     /* ... operations ... */
23
24     /* Single exit point ensures mutex release */
25 add_give_mutex:
26     xSemaphoreGive(manager->mutex);

```

```

27     return ret;
28 }

```

9.6 Error Interface Dependency Injection

Components receive error handlers through dependency injection.

```

1  /* Interface definition */
2  typedef struct star_error_interface {
3      esp_err_t (*record_error)(void* ctx, esp_err_t error,
4                               const char* message, const char* file,
5                               int line, const char* func);
6      bool (*can_retry)(void* ctx);
7      esp_err_t (*reset_state)(void* ctx);
8      void* ctx;
9  } star_error_interface_t;
10
11 /* Injecting error handler into component */
12 esp_err_t star_bus_manager_init(star_bus_manager_t* manager,
13                                 const char* tag,
14                                 star_error_interface_t* error_iface,
15                                 star_pin_interface_t* pin_iface)
16 {
17     manager->error_iface = error_iface; /* Can be NULL */
18     /* ... */
19 }
20
21 /* Using the error interface */
22 if (ret != ESP_OK) {
23     if (manager->error_iface && manager->error_iface->record_error) {
24         manager->error_iface->record_error(
25             manager->error_iface->ctx,
26             ret,
27             "Bus initialization failed",
28             __FILE__,
29             __LINE__,
30             __func__);
31     }
32 }

```

9.7 Cleanup Patterns

Use goto for consistent cleanup when multiple resources are acquired.

```
1 esp_err_t complex_init(complex_t* obj)
2 {
3     esp_err_t ret = ESP_OK;
4
5     /* Allocate resources */
6     obj->buffer = malloc(BUFFER_SIZE);
7     if (!obj->buffer) {
8         ret = ESP_ERR_NO_MEM;
9         goto cleanup;
10    }
11
12    obj->mutex = xSemaphoreCreateMutex();
13    if (!obj->mutex) {
14        ret = ESP_ERR_NO_MEM;
15        goto cleanup;
16    }
17
18    ret = hardware_init(obj);
19    if (ret != ESP_OK) {
20        goto cleanup;
21    }
22
23    return ESP_OK;
24
25 cleanup:
26    /* Release resources in reverse order */
27    if (obj->mutex) {
28        vSemaphoreDelete(obj->mutex);
29        obj->mutex = NULL;
30    }
31    if (obj->buffer) {
32        free(obj->buffer);
33        obj->buffer = NULL;
```

```

34     }
35     return ret;
36 }

```

9.8 Error Logging

Use appropriate log levels for different error severities.

```

1  /* ESP_LOGE - Errors that prevent operation */
2  ESP_LOGE(s_TAG, "Failed to initialize bus '%s': %s",
3           config->name, esp_err_to_name(ret));
4
5  /* ESP_LOGW - Warnings that don't prevent operation */
6  ESP_LOGW(manager->tag, "Failed to register pins for bus '%s': %s "
7           "(bus still added)", config->name, esp_err_to_name(ret));
8
9  /* ESP_LOGI - Informational messages */
10 ESP_LOGI(manager->tag, "Added bus config: %s (Type: %s)",
11          config->name, star_bus_type_to_string(config->type));
12
13 /* ESP_LOGD - Debug messages (compiled out in release) */
14 ESP_LOGD(s_TAG, "Write Success: %zu bytes to '%s' (Addr 0x%02X)",
15          len, config->name, address);

```

9.9 Summary

- Return `esp_err_t` from all functions that can fail
- Use `ESP_RETURN_ON_FALSE` for parameter validation
- Use `ESP_GOTO_ON_ERROR` for cleanup patterns
- Always validate pointers before dereferencing
- Handle mutex timeouts as `ESP_ERR_TIMEOUT`
- Use dependency injection for error handlers
- Release resources in reverse order of acquisition
- Log errors with appropriate severity levels

10.0 Functions

This section establishes standards for function design, implementation, and documentation in the STAR firmware codebase.

10.1 Return Value Conventions

- If the name of a function is an action (or an imperative command), it should return an **error-code integer** (`esp_err_t`).
- If the name is a predicate (a function that returns true/false), it should return a **succeeded boolean**.

```
1  /* Action functions return esp_err_t */
2  esp_err_t star_bus_manager_init(...);
3  esp_err_t star_bus_manager_add_bus(...);
4  esp_err_t star_bus_config_destroy(...);
5
6  /* Predicate functions return bool */
7  bool error_handler_can_retry(error_handler_t* handler);
8  bool is_bus_initialized(const star_bus_config_t* config);
9
10 /* Getter functions return the value or pointer */
11 star_bus_config_t* star_bus_manager_find_bus(...);
12 const char* star_bus_type_to_string(star_bus_type_t type);
```

10.2 Private and Exposed Private Functions

10.2.1 Private Functions (static)

Functions internal to a single source file should be declared static with the `internal_` prefix.

```
1  /* In .c file - private static functions */
2  static esp_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
3                                             gpio_num_t          pin,
4                                             const char*         bus_name,
5                                             const char*         usage,
6                                             bool                 can_be_shared)
7  {
8      if (pin < 0 || pin >= GPIO_NUM_MAX) {
```

```

9         return ESP_OK; /* Not an error, pin is not used */
10    }
11    /* ... implementation ... */
12}
13
14static esp_err_t internal_i2c_execute_with_retry(const star_bus_config_t* config,
15                                                i2c_port_t                port,
16                                                i2c_cmd_handle_t         cmd,
17                                                const char*
18                                                operation_name)
19{
20    /* ... implementation ... */
21}
22
23/* Declare prototypes at top of file */
24static esp_err_t internal_register_bus_pin(...);
25static esp_err_t internal_unregister_bus_pin(...);
26static esp_err_t internal_register_config_pins(...);

```

10.2.2 Exposed Private Functions (priv_)

If a function cannot be static and must be exposed across files within a module (but not publicly), prefix its name with `priv_`.

```

1 /* In internal header (e.g., star_bus_internal.h) */
2 esp_err_t priv_star_bus_validate_config(const star_bus_config_t* config);
3 esp_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);
4 void      priv_star_bus_release_mutex(star_bus_manager_t* manager);

```

10.3 Parameter Validation

Validate all parameters at the start of public functions.

```

1 esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2                                   star_bus_config_t*  config)
3 {
4     /* Validate required parameters */
5     ESP_RETURN_ON_FALSE(manager && config,
6                         ESP_ERR_INVALID_ARG,

```

```

7         s_TAG,
8         "Manager or config pointer is NULL");
9
10    /* Validate config contents */
11    ESP_RETURN_ON_FALSE(config->name && strlen(config->name) > 0,
12                        ESP_ERR_INVALID_ARG,
13                        manager->tag,
14                        "Config name is NULL or empty");
15
16    /* Validate enum range */
17    ESP_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
18                        config->type < k_star_bus_type_count,
19                        ESP_ERR_INVALID_ARG,
20                        manager->tag,
21                        "Invalid bus type in config");
22
23    /* Validate state */
24    ESP_RETURN_ON_FALSE(manager->mutex,
25                        ESP_ERR_INVALID_STATE,
26                        manager->tag,
27                        "Manager mutex not initialized");
28
29    /* ... rest of implementation ... */
30 }
31
32 esp_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
33                             const char*          name,
34                             const uint8_t*        data,
35                             size_t                len,
36                             uint8_t               reg_addr,
37                             size_t                bytes_written)
38 {
39     /* Validate all required pointers */
40     ESP_RETURN_ON_FALSE(manager && name && data,
41                         ESP_ERR_INVALID_ARG,
42                         s_TAG,
43                         "Manager, name, or data is NULL");
44

```

```

45  /* Validate numeric constraints */
46  ESP_RETURN_ON_FALSE(len > 0, ESP_ERR_INVALID_ARG, s_TAG,
47                      "Write length must be > 0");
48
49  /* Clear output parameters initially */
50  if (bytes_written) {
51      *bytes_written = 0;
52  }
53
54  /* ... rest of implementation ... */
55  }

```

10.4 Function Declaration Formatting

Align parameters vertically when they span multiple lines.

```

1  /* CORRECT - Aligned parameters */
2  esp_err_t star_bus_manager_init(star_bus_manager_t*    manager,
3                                  const char*            tag,
4                                  star_error_interface_t* error_iface,
5                                  star_pin_interface_t*   pin_iface);
6
7  star_bus_config_t* star_bus_config_create_spi_device(
8      const char*            name,
9      spi_host_device_t      host,
10     gpio_num_t             copi_pin,
11     gpio_num_t             cipo_pin,
12     gpio_num_t             sclk_pin,
13     int32_t                dma_chan,
14     const spi_device_interface_config_t* dev_cfg);
15
16  /* Function pointers in structs */
17  typedef struct star_error_interface {
18      esp_err_t (*record_error)(void*    ctx,
19                                  esp_err_t error,
20                                  const char* message,
21                                  const char* file,
22                                  int      line,
23                                  const char* func);

```

```

24     bool (*can_retry)(void* ctx);
25     esp_err_t (*reset_state)(void* ctx);
26     void* ctx;
27 } star_error_interface_t;

```

10.5 Doxygen Documentation

All public functions must have complete Doxygen documentation in header files.

```

1 /**
2  * @brief Add a new bus/device configuration to the manager.
3  *
4  * Adds a previously created configuration (via star_bus_config_create_*)
5  * to the manager's internal list. The manager takes ownership conceptually,
6  * but the configuration still needs to be destroyed eventually (typically via
7  * star_bus_manager_remove_bus or star_bus_manager_deinit).
8  *
9  * The configuration must have a unique name. This function is thread-safe.
10 *
11 * @param[in] manager Pointer to the initialized bus manager.
12 *                  Must not be NULL.
13 * @param[in] config Pointer to the bus configuration structure to add.
14 *                  Must not be NULL and must have a valid name.
15 * @return esp_err_t ESP_OK on success,
16 *          ESP_ERR_INVALID_ARG if manager or config is invalid,
17 *          ESP_ERR_INVALID_STATE if a bus with the same name
18 *          already exists or mutex error,
19 *          ESP_ERR_TIMEOUT if mutex times out.
20 */
21 esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
22                                   star_bus_config_t* config);

```

10.6 Private Function Documentation

Static functions should have brief documentation in the source file.

```

1 /**
2  * @brief Helper: Execute I2C command with retry logic
3  *

```

```

4  * Wraps i2c_master_cmd_begin() with automatic retry for transient errors.
5  *
6  * @param[in] config      Pointer to I2C bus configuration
7  * @param[in] port        I2C port number
8  * @param[in] cmd         I2C command handle
9  * @param[in] operation_name Name of operation for logging
10 * @return esp_err_t Final result after retries
11 */
12 static esp_err_t internal_i2c_execute_with_retry(const star_bus_config_t* config,
13                                                  i2c_port_t                port,
14                                                  i2c_cmd_handle_t         cmd,
15                                                  const char*
16                                                  operation_name);

```

10.7 Function Implementation Patterns

10.7.1 Single Exit Point with Cleanup

```

1  esp_err_t star_bus_manager_remove_bus(star_bus_manager_t* manager,
2                                       const char*          name)
3  {
4      ESP_RETURN_ON_FALSE(manager && name && strlen(name) > 0,
5                          ESP_ERR_INVALID_ARG, s_TAG, "Invalid arguments");
6
7      star_bus_config_t* to_remove = NULL;
8      star_bus_config_t* prev      = NULL;
9      esp_err_t          ret       = ESP_ERR_NOT_FOUND;
10
11     /* Acquire mutex */
12     if (xSemaphoreTake(manager->mutex,
13                         pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
14         != pdTRUE) {
15         return ESP_ERR_TIMEOUT;
16     }
17
18     /* Find and unlink */
19     for (star_bus_config_t* curr = manager->buses; curr;
20         prev = curr, curr = curr->next) {

```

```

21     if (curr->name && strcmp(curr->name, name) == 0) {
22         to_remove = curr;
23         if (prev == NULL) {
24             manager->buses = curr->next;
25         } else {
26             prev->next = curr->next;
27         }
28         to_remove->next = NULL;
29         ret = ESP_OK;
30         break;
31     }
32 }
33
34 if (ret == ESP_ERR_NOT_FOUND) {
35     goto remove_give_mutex;
36 }
37
38 /* Cleanup operations */
39 esp_err_t destroy_result = star_bus_config_destroy(to_remove);
40 if (destroy_result != ESP_OK) {
41     ret = destroy_result;
42 }
43
44 remove_give_mutex:
45     xSemaphoreGive(manager->mutex);
46     return ret;
47 }

```

10.7.2 Factory Function Pattern

```

1 star_bus_config_t* star_bus_config_create_i2c(const char* name,
2                                             i2c_port_t port,
3                                             uint8_t address,
4                                             gpio_num_t sda_pin,
5                                             gpio_num_t scl_pin,
6                                             uint32_t clk_speed)
7 {
8     if (name == NULL || strlen(name) == 0) {

```

```

9     ESP_LOGE(s_TAG, "Invalid name for I2C bus config");
10    return NULL;
11 }
12
13 star_bus_config_t* config = calloc(1, sizeof(star_bus_config_t));
14 if (config == NULL) {
15     ESP_LOGE(s_TAG, "Failed to allocate memory for I2C config");
16     return NULL;
17 }
18
19 /* Initialize common fields */
20 config->name          = strdup(name);
21 config->type          = k_star_bus_type_i2c;
22 config->initialized    = false;
23 config->handle        = NULL;
24 config->next          = NULL;
25
26 /* Initialize I2C-specific fields */
27 config->proto.i2c.port    = port;
28 config->proto.i2c.address = address;
29 config->proto.i2c.config.sda_io_num = sda_pin;
30 config->proto.i2c.config.scl_io_num = scl_pin;
31 config->proto.i2c.config.master.clk_speed = clk_speed;
32
33 /* Set default operations */
34 star_bus_i2c_init_default_ops(&config->proto.i2c.ops);
35
36 return config;
37 }

```

10.8 Callback Functions

```

1 /* Callback type definition */
2 typedef esp_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
3                                           void* user_ctx);
4
5 /* Function using callback */
6 esp_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,

```

```

7         const char*      name,
8         star_bus_callback_t callback,
9         void*            user_ctx)
10    {
11        ESP_RETURN_ON_FALSE(manager && name && callback,
12                             ESP_ERR_INVALID_ARG, s_TAG, "Invalid arguments");
13
14        /* ... find bus and invoke callback while holding mutex ... */
15        esp_err_t callback_result = callback(found_bus, user_ctx);
16
17        xSemaphoreGive(manager->mutex);
18        return callback_result;
19    }
20
21    /* Using the callback pattern */
22    esp_err_t my_bus_callback(star_bus_config_t* bus, void* ctx) {
23        ESP_LOGI(TAG, "Bus type: %s", star_bus_type_to_string(bus->type));
24        return ESP_OK;
25    }
26
27    star_bus_manager_with_bus(&manager, "sensor_i2c", my_bus_callback, NULL);

```

10.9 Summary

- Action functions return `esp_err_t`; predicates return `bool`
- Use `static` with `internal_` prefix for file-local functions
- Use `priv_` prefix for exposed-but-not-public functions
- Validate all parameters at function start
- Align parameters vertically in multi-line declarations
- Document all public functions with complete Doxygen
- Use single exit point with cleanup for complex functions
- Use factory functions for object creation

11.0 Global Variables

Global variables should be avoided when possible. This section establishes patterns for when they are necessary.

11.1 General Principles

- **Avoid Globals:** Global variables are strongly discouraged. Use dependency injection instead.
- **File-Scoped When Necessary:** If a global is needed, limit its scope to a single file using `static`.
- **Use Prefix Conventions:** `s_` for static file-scoped, `g_` for true globals.
- **Prefer `static const` Over `#define`:** For typed constants, use `static const`.

11.2 Static File-Scoped Variables (`s_` prefix)

Use `static` with `s_` prefix for variables that should be accessible only within a single source file.

```
1  /* CORRECT - Static file-scoped variables */
2
3  /* Logging tag - most common pattern */
4  static const char* s_TAG = "STAR_BUS_MANAGER";
5
6  /* Configuration constants - prefer static const over #define */
7  static const uint32_t s_i2c_timeout_ms = 1000;
8  static const uint32_t s_default_retry_count = 3;
9
10 /* State variables scoped to this file */
11 static bool s_module_initialized = false;
12
13 /* From star_bus_i2c.c */
14 static const char* s_TAG = "STAR_BUS_I2C";
15 static const uint32_t s_i2c_timeout_ms = 1000;
16
17 /* From star_bus_config.c */
18 static const char* s_TAG = "STAR_BUS_CONFIG";
```

11.3 Why `static const` Over `#define`

`static const` provides type safety and scope control that `#define` lacks.

```
1  /* PREFERRED - Type-safe, scoped, debugger-visible */
2  static const uint32_t s_timeout_ms = 1000;
3  static const size_t   s_buffer_size = 256;
```

```

4
5 /* Use the constant with type checking */
6 uint32_t timeout = s_timeout_ms; /* Compiler checks type */
7
8 /* AVOID - No type safety, global namespace pollution */
9 #define TIMEOUT_MS 1000
10 #define BUFFER_SIZE 256
11
12 /* Issues with #define: */
13 /* 1. No type checking */
14 /* 2. Can collide with other defines */
15 /* 3. Not visible in debugger */
16 /* 4. Can have unexpected macro expansion */

```

11.4 When to Use #define

Use #define for:

- Compile-time constants for array sizes
- Conditional compilation
- Macros that cannot be functions

```

1 /* Array sizes must be compile-time constants */
2 #define STAR_BUS_NAME_MAX_LEN (32)
3
4 char bus_name[STAR_BUS_NAME_MAX_LEN]; /* OK - compile-time constant */
5
6 /* Conditional compilation */
7 #ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING
8     ESP_LOGD(s_TAG, "Debug info: %d", value);
9 #endif
10
11 /* Macros with special features */
12 #define RECORD_ERROR(handler, code, desc) \
13     do { \
14         error_handler_record_error((handler), (code), (desc), \
15             __FILE__, __LINE__, __func__); \
16     } while (0)

```

11.5 True Global Variables (g_ prefix)

If a truly global variable is unavoidable, use the g_ prefix. This should be rare.

```
1  /* DISCOURAGED but sometimes necessary */
2  /* g_ prefix for true globals (visible across files) */
3  g_system_state_t g_system_state;
4
5  /* Prefer dependency injection instead */
6  /* BETTER approach: */
7  typedef struct app_context {
8      star_bus_manager_t  bus_manager;
9      error_handler_t     error_handler;
10     /* ... */
11 } app_context_t;
12
13 /* Pass context to functions that need it */
14 void app_process_sensors(app_context_t* ctx);
```

11.6 Dependency Injection Alternative

Replace globals with dependency injection.

```
1  /* WRONG - Global error handler */
2  error_handler_t g_error_handler; /* Global - bad */
3
4  void some_function(void) {
5      if (error) {
6          RECORD_ERROR(&g_error_handler, err, "Failed"); /* Uses global */
7      }
8  }
9
10 /* CORRECT - Dependency injection */
11 typedef struct star_bus_manager {
12     star_error_interface_t* error_iface; /* Injected dependency */
13     /* ... */
14 } star_bus_manager_t;
15
16 esp_err_t star_bus_manager_init(star_bus_manager_t* manager,
17                                const char* tag,
```

```

18             star_error_interface_t* error_iface, /* Injected
19             */
20             star_pin_interface_t* pin_iface) /* Injected
21             */
22     {
23         manager->error_iface = error_iface;
24         /* ... */
25     }
26
27     /* Usage */
28     error_handler_t error_handler;
29     error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
30
31     star_error_interface_t error_iface;
32     error_handler_get_interface(&error_iface, &error_handler);
33
34     star_bus_manager_t bus_manager;
35     star_bus_manager_init(&bus_manager, "main", &error_iface, NULL);

```

11.7 Static Variables for Module State

When a module needs to maintain state, prefer explicit initialization and deinitialization.

```

1  /* Module-private state */
2  static struct {
3      bool    initialized;
4      uint32_t transaction_count;
5      /* ... */
6  } s_module_state = {
7      .initialized = false,
8      .transaction_count = 0,
9  };
10
11  esp_err_t module_init(void)
12  {
13      if (s_module_state.initialized) {
14          return ESP_ERR_INVALID_STATE;
15      }
16

```

```

17     /* Initialize module */
18     s_module_state.initialized = true;
19     return ESP_OK;
20 }
21
22 esp_err_t module_deinit(void)
23 {
24     if (!s_module_state.initialized) {
25         return ESP_ERR_INVALID_STATE;
26     }
27
28     /* Clean up */
29     s_module_state.initialized = false;
30     s_module_state.transaction_count = 0;
31     return ESP_OK;
32 }

```

11.8 Thread Safety for Shared State

If static variables are accessed from multiple threads, protect with mutex.

```

1  /* Thread-safe module state */
2  static struct {
3      bool            initialized;
4      SemaphoreHandle_t mutex;
5      uint32_t        counter;
6  } s_shared_state;
7
8  esp_err_t safe_increment_counter(void)
9  {
10     if (xSemaphoreTake(s_shared_state.mutex, pdMS_TO_TICKS(1000)) != pdTRUE) {
11         return ESP_ERR_TIMEOUT;
12     }
13
14     s_shared_state.counter++;
15
16     xSemaphoreGive(s_shared_state.mutex);
17     return ESP_OK;
18 }

```

11.9 Summary

Type	Prefix	Usage
Static file-scoped	s_	Prefer this for module-internal state
True global	g_	Avoid; use dependency injection
static const	s_	Type-safe constants
#define	None	Array sizes, macros, conditional compilation

- Use `static const` for type-safe constants
- Use `s_TAG` for module logging tags
- Prefer dependency injection over global variables
- Protect shared state with mutexes
- Limit variable scope to the smallest necessary

12.0 Header Files

This section establishes standards for header file organization and content in the STAR firmware codebase.

12.1 General Principles

- **Header Files Should Only Contain Declarations:** No implementations except for inline functions.
- **Minimal Includes in Headers:** Include only what is necessary for the declarations.
- **Self-Contained Headers:** Each header should compile independently.
- **One Purpose Per Header:** Split large headers into focused sub-headers.

12.2 Include Guard Naming Convention

All header files must use include guards following the pattern: `STAR_COMPONENT_FILENAME_H`

```
1 /* CORRECT - from star_bus_manager.h */
2 #ifndef STAR_COMPONENT_BUS_MANAGER_H
3 #define STAR_COMPONENT_BUS_MANAGER_H
4
5 /* ... declarations ... */
```

```

6
7 #endif /* STAR_COMPONENT_BUS_MANAGER_H */
8
9 /* CORRECT - from star_bus_config.h */
10 #ifndef STAR_COMPONENT_BUS_CONFIG_H
11 #define STAR_COMPONENT_BUS_CONFIG_H
12 /* ... */
13 #endif /* STAR_COMPONENT_BUS_CONFIG_H */
14
15 /* CORRECT - from star_error_interface.h */
16 #ifndef STAR_ERROR_INTERFACE_H
17 #define STAR_ERROR_INTERFACE_H
18 /* ... */
19 #endif /* STAR_ERROR_INTERFACE_H */
20
21 /* CORRECT - from star_pin_interface.h */
22 #ifndef STAR_PIN_INTERFACE_H
23 #define STAR_PIN_INTERFACE_H
24 /* ... */
25 #endif /* STAR_PIN_INTERFACE_H */

```

12.3 C++ Compatibility

All headers must include C++ extern “C” blocks for compatibility with C++ compilers.

```

1 #ifndef STAR_COMPONENT_BUS_MANAGER_H
2 #define STAR_COMPONENT_BUS_MANAGER_H
3
4 #ifdef __cplusplus
5 extern "C" {
6 #endif
7
8 /* All declarations go here */
9
10 #ifdef __cplusplus
11 }
12 #endif
13
14 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

12.4 Header File Structure

Headers should follow this organization:

```
1  /* lib/star_bus/include/star_bus_manager.h */
2
3  #ifndef STAR_COMPONENT_BUS_MANAGER_H
4  #define STAR_COMPONENT_BUS_MANAGER_H
5
6  #ifdef __cplusplus
7  extern "C" {
8  #endif
9
10 /* === 1. Includes === */
11 #include "esp_err.h"
12 #include "star_bus_manager_types.h"
13
14 /* === 2. File Documentation === */
15 /**
16  * @file star_bus_manager.h
17  * @brief Unified bus manager for I2C, SPI, GPIO, and other bus protocols
18  *
19  * The bus manager provides a central registry for managing multiple
20  * bus configurations. It supports dependency injection of error handlers
21  * and pin validators for loose coupling.
22  *
23  * Example Usage:
24  * @code
25  * // Initialize bus manager with dependency injection
26  * star_bus_manager_t bus_manager;
27  * star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
28  * @endcode
29  */
30
31 /* === 3. Forward Declarations (if needed) === */
32 /* Forward declarations reduce header dependencies */
33
```

```

34  /* === 4. Macros and Constants === */
35  #define STAR_BUS_NAME_MAX_LEN (32)
36
37  /* === 5. Type Definitions === */
38  /* (Usually in separate _types.h header) */
39
40  /* === 6. Function Declarations === */
41
42  /**
43   * @brief Initialize a bus manager instance.
44   *
45   * @param[out] manager      Pointer to bus manager to initialize.
46   * @param[in]  tag          Optional tag string for logging.
47   * @param[in]  error_iface  Optional error handler interface.
48   * @param[in]  pin_iface    Optional pin validator interface.
49   * @return esp_err_t ESP_OK on success.
50   */
51  esp_err_t star_bus_manager_init(star_bus_manager_t*  manager,
52                                  const char*          tag,
53                                  star_error_interface_t* error_iface,
54                                  star_pin_interface_t*  pin_iface);
55
56  /* ... more function declarations ... */
57
58  #ifdef __cplusplus
59  }
60  #endif
61
62  #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

12.5 Include Order

Includes should be ordered in groups, separated by blank lines:

```

1  /* 1. Primary header (in .c file, include corresponding .h first) */
2  #include "star_bus_manager.h"
3
4  /* 2. FreeRTOS includes */
5  #include "freertos/FreeRTOS.h"

```

```

6 #include "freertos/semphr.h"
7 #include "freertos/task.h"
8
9 /* 3. Standard library includes */
10 #include <inttypes.h>
11 #include <stdbool.h>
12 #include <stddef.h>
13 #include <stdint.h>
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <string.h>
17
18 /* 4. ESP-IDF includes */
19 #include "driver/gpio.h"
20 #include "driver/i2c.h"
21 #include "esp_check.h"
22 #include "esp_err.h"
23 #include "esp_log.h"
24 #include "sdkconfig.h"
25
26 /* 5. Project includes */
27 #include "star_bus_config.h"
28 #include "star_bus_i2c.h"
29 #include "star_bus_types.h"
30 #include "star_error_interface.h"

```

12.6 Forward Declarations

Use forward declarations to minimize header dependencies.

```

1 /* In star_bus_config.h - forward declare instead of including */
2 /* Forward declaration */
3 struct star_bus_manager; /* Don't need full definition */
4
5 /* Function using forward-declared type */
6 esp_err_t star_bus_config_init(star_bus_config_t*      config,
7                                struct star_bus_manager* manager);
8
9 /* In star_bus_common_types.h */

```

```

10  /* Forward declarations for pointer-only usage */
11  typedef struct star_bus_config  star_bus_config_t;
12  typedef struct star_bus_manager star_bus_manager_t;

```

12.7 Separating Types from Functions

For complex modules, split type definitions into dedicated headers.

```

1  /* File organization for star_bus module */
2  star_bus/include/
3      star_bus_manager.h      /* Main API (functions) */
4      star_bus_manager_types.h /* Types for manager */
5      star_bus_common_types.h /* Shared types */
6      star_bus_protocol_types.h /* Protocol-specific types */
7      star_bus_function_types.h /* Function pointer types */
8      star_bus_event_types.h  /* Event structures */
9      star_bus_types.h        /* Controller - includes all */
10
11  /* Controller header pattern */
12  /* star_bus_types.h */
13  #ifndef STAR_COMPONENT_BUS_TYPES_H
14  #define STAR_COMPONENT_BUS_TYPES_H
15
16  #ifdef __cplusplus
17  extern "C" {
18  #endif
19
20  /* Include all type headers */
21  #include "star_bus_common_types.h"
22  #include "star_bus_event_types.h"
23  #include "star_bus_function_types.h"
24  #include "star_bus_manager_types.h"
25  #include "star_bus_protocol_types.h"
26
27  #ifdef __cplusplus
28  }
29  #endif
30
31  #endif /* STAR_COMPONENT_BUS_TYPES_H */

```

12.8 Doxygen File Documentation

Include @file documentation for headers with significant content.

```
1 /**
2  * @file star_error_handler.h
3  * @brief Error handling with retry logic and exponential backoff
4  *
5  * This module provides a concrete implementation of the error interface
6  * with automatic retry management, exponential backoff delays, and
7  * optional reset function callbacks.
8  *
9  * Key Features:
10 * - Configurable max retries and backoff delays
11 * - Exponential backoff between retries
12 * - Optional custom reset function for recovery
13 * - Thread-safe with mutex protection
14 *
15 * Example Usage:
16 * @code
17 * error_handler_t error_handler;
18 * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
19 *
20 * if (operation_failed()) {
21 *     RECORD_ERROR(&error_handler, ESP_ERR_TIMEOUT, "Timeout");
22 * }
23 * @endcode
24 */
```

12.9 Complete Header Template

```
1 /* lib/star_module/include/star_module.h */
2
3 #ifndef STAR_COMPONENT_MODULE_H
4 #define STAR_COMPONENT_MODULE_H
5
```

```

6  #ifdef __cplusplus
7  extern "C" {
8  #endif
9
10 /* === Includes === */
11 #include <stdbool.h>
12 #include <stdint.h>
13
14 #include "esp_err.h"
15
16 /* === File Documentation === */
17 /**
18  * @file star_module.h
19  * @brief Brief description of this module
20  *
21  * Detailed description of what this module provides and how to use it.
22  */
23
24 /* === Forward Declarations === */
25 typedef struct star_module star_module_t;
26
27 /* === Macros === */
28 #define STAR_MODULE_DEFAULT_TIMEOUT_MS (1000)
29
30 /* === Type Definitions === */
31 typedef struct star_module {
32     bool        initialized;
33     uint32_t     timeout_ms;
34     void*        user_ctx;
35 } star_module_t;
36
37 /* === Function Declarations === */
38
39 /**
40  * @brief Initialize the module.
41  *
42  * @param[out] module Pointer to module structure to initialize.
43  * @param[in]  timeout Timeout in milliseconds.

```

```

44  * @return esp_err_t ESP_OK on success.
45  */
46  esp_err_t star_module_init(star_module_t* module, uint32_t timeout);
47
48  /**
49   * @brief Deinitialize the module.
50   *
51   * @param[in,out] module Pointer to module structure to deinitialize.
52   * @return esp_err_t ESP_OK on success.
53   */
54  esp_err_t star_module_deinit(star_module_t* module);
55
56  #ifdef __cplusplus
57  }
58  #endif
59
60  #endif /* STAR_COMPONENT_MODULE_H */

```

12.10 Summary

- Use include guards: STAR_COMPONENT_FILENAME_H
- Include C++ extern “C” blocks
- Order includes: FreeRTOS, standard library, ESP-IDF, project
- Use forward declarations to minimize dependencies
- Split types into separate _types.h headers for complex modules
- Document files with @file Doxygen blocks
- Keep headers focused on declarations, not implementations

13.0 Horizontal Space

This section establishes standards for horizontal spacing in the STAR firmware codebase.

13.1 General Rules

- **Space After Keywords:** Add space after if, for, while, switch.
- **Binary Operators:** Add a single space around binary operators.

- **No Space After Function Names:** No space between function name and opening parenthesis.
- **Alignment:** Align related declarations and comments.

13.2 Space After Control Keywords

Add a space between keywords and opening parenthesis.

```

1  /* CORRECT - Space after keywords */
2  if (ret != ESP_OK) {
3      return ret;
4  }
5
6  for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
7      process(curr);
8  }
9
10 while (retry_count > 0) {
11     attempt_operation();
12     retry_count--;
13 }
14
15 switch (config->type) {
16     case k_star_bus_type_i2c: {
17         /* ... */
18         break;
19     }
20 }
21
22 /* WRONG - No space after keywords */
23 if(ret != ESP_OK) {          /* Missing space */
24     return ret;
25 }
26
27 for(int i = 0; i < 10; i++) { /* Missing space */
28     /* ... */
29 }

```

13.3 No Space After Function Names

Do not add space between function name and opening parenthesis.

```
1  /* CORRECT - No space after function name */
2  esp_err_t ret = star_bus_manager_init(&manager, "main", NULL, NULL);
3  ESP_LOGI(s_TAG, "Initialized bus manager");
4  free(buffer);
5
6  /* WRONG - Space after function name */
7  esp_err_t ret = star_bus_manager_init (&manager, "main", NULL, NULL);
8  ESP_LOGI (s_TAG, "Initialized bus manager");
9  free (buffer);
```

13.4 Binary Operators

Add a single space around binary operators.

```
1  /* CORRECT - Spaces around binary operators */
2  int result = a + b;
3  bool is_valid = (count > 0) && (size <= MAX_SIZE);
4  uint32_t mask = flags | FLAG_ENABLED;
5  int shifted = value << 2;
6  size_t remaining = total - processed;
7
8  /* Assignment operators */
9  count += 1;
10 flags |= FLAG_INITIALIZED;
11 value >>= 4;
12
13 /* Comparison operators */
14 if (ret != ESP_OK) { /* ... */ }
15 if (count >= threshold) { /* ... */ }
16 if (ptr == NULL) { /* ... */ }
17
18 /* WRONG - Missing spaces */
19 int result = a+b;
20 bool is_valid = (count>0)&&(size<=MAX_SIZE);
21 if (ret!=ESP_OK) { /* ... */ }
```

13.5 Unary Operators

No space between unary operator and operand.

```
1 /* CORRECT - No space with unary operators */
2 count++;
3 --remaining;
4 bool is_null = !ptr;
5 int negative = -value;
6 uint32_t complement = ~mask;
7 size_t size = sizeof(star_bus_config_t);
8 int* ptr = &value;
9 int deref = *ptr;
10
11 /* WRONG - Space with unary operators */
12 count ++;
13 -- remaining;
14 bool is_null = ! ptr;
```

13.6 Pointer Declarations

Place asterisk with the type, not the variable name.

```
1 /* CORRECT - Asterisk with type */
2 star_bus_config_t* config;
3 const char* name;
4 void* user_ctx;
5
6 /* Function parameters */
7 esp_err_t star_bus_manager_init(star_bus_manager_t*    manager,
8                                const char*            tag,
9                                star_error_interface_t* error_iface,
10                               star_pin_interface_t*   pin_iface);
11
12 /* Multiple pointers on same line (avoid when possible) */
13 int *a, *b; /* Both are pointers */
14
15 /* WRONG - Various incorrect styles */
16 star_bus_config_t *config; /* Asterisk with variable */
17 star_bus_config_t * config; /* Spaces around asterisk */
```

13.7 Comma and Semicolon Spacing

Space after comma and semicolon, not before.

```
1  /* CORRECT */
2  star_bus_manager_init(&manager, "main", &error_iface, &pin_iface);
3
4  for (int i = 0; i < count; i++) {
5      /* ... */
6  }
7
8  int array[] = {1, 2, 3, 4, 5};
9
10 /* WRONG */
11 star_bus_manager_init(&manager , "main" , &error_iface , &pin_iface);
12 for (int i = 0 ; i < count ; i++) { /* ... */ }
13 int array[] = {1 , 2 , 3 , 4 , 5};
```

13.8 Parentheses Spacing

No space after opening parenthesis or before closing parenthesis.

```
1  /* CORRECT */
2  if (ret != ESP_OK) {
3      return ret;
4  }
5
6  result = calculate(a, b, c);
7
8  /* WRONG - Spaces inside parentheses */
9  if ( ret != ESP_OK ) {
10     return ret;
11 }
12
13 result = calculate( a, b, c );
```

13.9 Parameter Alignment

Align parameters vertically in multi-line function declarations and calls.

```
1  /* CORRECT - Aligned parameters */
2  esp_err_t star_bus_manager_init(star_bus_manager_t*    manager,
3                                  const char*            tag,
4                                  star_error_interface_t* error_iface,
5                                  star_pin_interface_t*   pin_iface);
6
7  star_bus_config_t* star_bus_config_create_spi_device(
8      const char*            name,
9      spi_host_device_t      host,
10     gpio_num_t             copi_pin,
11     gpio_num_t             cipo_pin,
12     gpio_num_t             sclk_pin,
13     int32_t                dma_chan,
14     const spi_device_interface_config_t* dev_cfg);
15
16  /* Long function calls */
17  ESP_RETURN_ON_FALSE(manager && config,
18                      ESP_ERR_INVALID_ARG,
19                      s_TAG,
20                      "Manager or config pointer is NULL");
```

13.10 Variable and Comment Alignment

Align related variables and their comments.

```
1  /* CORRECT - Aligned declarations and comments */
2  typedef struct error_handler {
3      uint32_t      error_count;          /**< Total errors recorded */
4      uint32_t      max_retries;          /**< Max retry attempts */
5      uint32_t      current_retry;        /**< Current retry counter */
6      uint32_t      base_retry_delay;     /**< Base delay (ms) */
7      uint32_t      max_retry_delay;      /**< Max delay (ms) */
8      uint32_t      current_retry_delay;  /**< Current delay with backoff */
9      esp_err_t     last_error;           /**< Last recorded error code */
10     bool           in_error_state;       /**< Whether in error state */
11     void*          reset_context;        /**< Context for reset function */
```

```

12     SemaphoreHandle_t mutex;                /**< Mutex for thread-safety */
13 } error_handler_t;
14
15 /* Aligned local variables */
16 i2c_port_t      port      = config->proto.i2c.port;
17 uint8_t         address   = config->proto.i2c.address;
18 esp_err_t       ret       = ESP_OK;
19 i2c_cmd_handle_t cmd      = NULL;

```

13.11 Struct Initializer Alignment

Align designated initializers.

```

1  /* CORRECT - Aligned initializers */
2  star_bus_event_t event = {
3      .bus_type = k_star_bus_type_i2c,
4      .bus_name = config->name,
5      .i2c      = {
6          .port      = port,
7          .address   = address,
8          .is_write  = true,
9          .len       = len
10     }
11 };
12
13 spi_device_interface_config_t spi_cfg = {
14     .clock_speed_hz = 10 * 1000 * 1000,
15     .mode           = 0,
16     .spics_io_num   = GPIO_NUM_5,
17     .queue_size     = 7,
18     .pre_cb         = NULL,
19     .post_cb        = NULL,
20 };

```

13.12 Summary

Context	Rule
After if, for, while, switch	Space
After function name	No space
Around binary operators	Space
With unary operators	No space
After comma/semicolon	Space
Inside parentheses	No space
Pointer asterisk	With type
Multi-line parameters	Align vertically

14.0 Include Statements

This section establishes standards for include statement organization and ordering in the STAR firmware codebase.

14.1 General Rules

- **Use Angle Brackets for System/Library Headers:** `<stdio.h>`.
- **Use Quotation Marks for Project Headers:** `"my_project.h"`.
- **Minimal Includes in Header Files:** Only include what is necessary.
- **Group and Order Includes:** Follow consistent ordering.

14.2 Include Order

Includes should be ordered in these groups, separated by blank lines:

1. Primary header (in .c files)
2. FreeRTOS includes
3. Standard library includes
4. ESP-IDF includes
5. Project includes

```
1 /* lib/star_bus/src/star_bus_i2c.c */
2
3 /* 1. Primary header - always first in .c file */
4 #include "star_bus_i2c.h"
```

```

5
6  /* 2. FreeRTOS includes */
7  #include "freertos/FreeRTOS.h"
8
9  /* 3. Standard library includes */
10 #include <inttypes.h>
11 #include <string.h>
12
13 /* 4. ESP-IDF includes */
14 #include "esp_check.h"
15 #include "esp_log.h"
16
17 /* 5. Project includes */
18 #include "star_bus_manager.h"
19 #include "star_bus_types.h"

```

14.3 Complete Example from Codebase

```

1  /* lib/star_bus/src/star_bus_manager.c */
2
3  /* Primary header */
4  #include "star_bus_manager.h"
5
6  /* FreeRTOS */
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/semphr.h"
9
10 /* Standard library */
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14
15 /* ESP-IDF */
16 #include "esp_check.h"
17 #include "esp_log.h"
18 #include "sdkconfig.h"
19
20 /* Project includes */

```

```
21 #include "star_bus_adc.h"
22 #include "star_bus_config.h"
23 #include "star_bus_gpio.h"
24 #include "star_bus_i2c.h"
25 #include "star_bus_spi.h"
```

14.4 FreeRTOS Include Order

FreeRTOS requires specific include ordering. Always include FreeRTOS.h first.

```
1 /* CORRECT - FreeRTOS.h must come first */
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/semphr.h"
4 #include "freertos/task.h"
5 #include "freertos/queue.h"
6 #include "freertos/event_groups.h"
7
8 /* WRONG - Other FreeRTOS headers before FreeRTOS.h */
9 #include "freertos/semphr.h" /* Error: FreeRTOS.h not included first */
10 #include "freertos/FreeRTOS.h"
```

14.5 System Includes

Use angle brackets for system and standard library headers.

```
1 /* Standard C library */
2 #include <inttypes.h>
3 #include <stdbool.h>
4 #include <stddef.h>
5 #include <stdint.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
```

14.6 ESP-IDF Includes

ESP-IDF headers use quotation marks but are treated as a separate group.

```

1  /* ESP-IDF driver includes */
2  #include "driver/gpio.h"
3  #include "driver/i2c.h"
4  #include "driver/spi_master.h"
5  #include "driver/uart.h"
6
7  /* ESP-IDF system includes */
8  #include "esp_check.h"
9  #include "esp_err.h"
10 #include "esp_log.h"
11 #include "esp_system.h"
12
13 /* ESP-IDF ADC includes */
14 #include "esp_adc/adc_cali.h"
15 #include "esp_adc/adc_cali_scheme.h"
16 #include "esp_adc/adc_oneshot.h"
17
18 /* Configuration */
19 #include "sdkconfig.h"

```

14.7 Project Includes

Use quotation marks for project headers. Alphabetize within each group.

```

1  /* Project includes - alphabetized */
2  #include "star_bus_adc.h"
3  #include "star_bus_config.h"
4  #include "star_bus_gpio.h"
5  #include "star_bus_i2c.h"
6  #include "star_bus_manager.h"
7  #include "star_bus_spi.h"
8  #include "star_bus_types.h"
9  #include "star_error_interface.h"
10 #include "star_pin_interface.h"

```

14.8 Header File Includes

Header files should minimize includes and use forward declarations when possible.

```

1  /* star_bus_manager.h - Header file includes */
2  #ifndef STAR_COMPONENT_BUS_MANAGER_H
3  #define STAR_COMPONENT_BUS_MANAGER_H
4
5  #ifdef __cplusplus
6  extern "C" {
7  #endif
8
9  /* Only include what's needed for declarations */
10 #include "esp_err.h"
11 #include "star_bus_manager_types.h"
12
13 /* Use forward declarations for pointer-only usage */
14 struct star_bus_config; /* Don't include star_bus_config.h */
15
16 /* Function using forward-declared type */
17 esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
18                                     struct star_bus_config* config);
19
20 #ifdef __cplusplus
21 }
22 #endif
23
24 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

14.9 Conditional Includes

Use `#ifdef` for platform-specific or optional includes.

```

1  /* Platform-specific includes */
2  #ifdef CONFIG_IDF_TARGET_ESP32S3
3  #include "esp32s3/rom/gpio.h"
4  #else
5  #include "esp32/rom/gpio.h"
6  #endif
7
8  /* Optional feature includes */
9  #ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING

```

```
10 #include "esp_debug_helpers.h"
11 #endif
```

14.10 Include Comments

Add comments for non-obvious includes explaining why they are needed.

```
1 #include "star_bus_manager.h"
2
3 #include "freertos/FreeRTOS.h"
4
5 #include <inttypes.h>
6 #include <string.h>
7
8 #include "esp_check.h"
9 #include "esp_log.h"
10 #include "star_bus_manager.h" /* Needed for star_bus_manager_find_bus */
11 #include "star_bus_types.h" /* Needed for star_bus_config_t and union */
```

14.11 Avoiding Circular Dependencies

Structure your includes to avoid circular dependencies.

```
1 /* Problem: Circular dependency */
2 /* star_bus_manager.h includes star_bus_config.h */
3 /* star_bus_config.h includes star_bus_manager.h */
4
5 /* Solution: Use forward declarations */
6 /* star_bus_config.h */
7 struct star_bus_manager; /* Forward declare, don't include */
8
9 esp_err_t star_bus_config_init(star_bus_config_t* config,
10                               struct star_bus_manager* manager);
11
12 /* star_bus_config.c - Include full header here */
13 #include "star_bus_config.h"
14 #include "star_bus_manager.h" /* Full definition needed in .c */
```

14.12 Summary

Order	Category	Style
1	Primary header (.c files)	"..."
2	FreeRTOS	"freertos/..."
3	Standard library	<...>
4	ESP-IDF	"..."
5	Project	"..."

- Always include `FreeRTOS.h` before other FreeRTOS headers
- Separate groups with blank lines
- Alphabetize within each group
- Minimize includes in headers; use forward declarations
- Add comments for non-obvious include dependencies

15.0 Indentation

This project follows the rule of **2 spaces** for indentation, without using tabs.

16.0 Line Endings

Unix-style line endings ('LF', represented as '\n') are mandatory.

17.0 Logging

Logging is handled using the ESP-IDF logging macros.

- **ESP_LOGE** - Error
- **ESP_LOGW** - Warning
- **ESP_LOGI** - Info
- **ESP_LOGD** - Debug
- **ESP_LOGV** - Verbose

18.0 Naming Conventions

This section establishes comprehensive naming standards for all identifiers in the STAR firmware codebase. Consistent naming improves readability, maintainability, and reduces cognitive load

when navigating the codebase.

18.1 General Rules

- **Language:** All identifiers must be in English.
- **Clarity:** Names should be descriptive and self-documenting.
- **Abbreviations:** Avoid abbreviations unless they are widely understood (e.g., GPIO, I2C, SPI).

18.2 Functions and Variables

Functions and variables use `snake_case`—all lowercase with words separated by underscores.

```
1 /* CORRECT - snake_case for functions and variables */
2 esp_err_t star_bus_manager_init(star_bus_manager_t* manager, const char* tag);
3 uint32_t error_count;
4 size_t bytes_written;
5 bool is_initialized;
6
7 /* WRONG - other naming styles */
8 esp_err_t StarBusManagerInit(); /* PascalCase */
9 esp_err_t starBusManagerInit(); /* camelCase */
10 uint32_t ErrorCount; /* PascalCase */
```

18.3 Module Prefixes

All public functions must use the module prefix pattern: `star_[module]_[function]`.

```
1 /* From star_bus_manager.h */
2 esp_err_t star_bus_manager_init(...);
3 esp_err_t star_bus_manager_add_bus(...);
4 esp_err_t star_bus_manager_remove_bus(...);
5 esp_err_t star_bus_manager_deinit(...);
6
7 /* From star_bus_config.h */
8 star_bus_config_t* star_bus_config_create_i2c(...);
9 star_bus_config_t* star_bus_config_create_spi_device(...);
10 esp_err_t star_bus_config_destroy(...);
11
```

```

12 /* From star_bus_i2c.h */
13 esp_err_t star_bus_i2c_write(...);
14 esp_err_t star_bus_i2c_read(...);
15 void      star_bus_i2c_init_default_ops(...);
16
17 /* From star_error_handler.h */
18 esp_err_t error_handler_init(...);
19 esp_err_t error_handler_record_error(...);
20 bool      error_handler_can_retry(...);

```

18.4 Constants and Macros

Constants and macros use SCREAMING_SNAKE_CASE—all uppercase with words separated by underscores.

```

1 /* CORRECT - SCREAMING_SNAKE_CASE for macros and constants */
2 #define STAR_BUS_NAME_MAX_LEN          (32)
3 #define CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS  (1000)
4 #define I2C_MASTER_WRITE                0
5 #define I2C_MASTER_READ                 1
6 #define GPIO_NUM_MAX                    48
7
8 /* Convenience macro with proper naming */
9 #define RECORD_ERROR(handler, code, desc) \
10     do { \
11         error_handler_record_error((handler), (code), (desc), \
12             __FILE__, __LINE__, __func__); \
13     } while (0)
14
15 #define STAR_IFACE_RECORD_ERROR(iface, err, msg) \
16     ((iface) && (iface)->record_error \
17         ? (iface)->record_error((iface)->ctx, (err), (msg), \
18             __FILE__, __LINE__, __func__) \
19         : (err))
20
21 /* WRONG */
22 #define star_bus_name_max_len  32  /* lowercase */
23 #define StarBusNameMaxLen      32  /* PascalCase */

```

18.5 Type Names

All custom types must use snake_case with the _t suffix.

```
1  /* CORRECT - snake_case_t for type names */
2  typedef struct star_bus_config  star_bus_config_t;
3  typedef struct star_bus_manager star_bus_manager_t;
4  typedef struct error_handler    error_handler_t;
5  typedef enum star_bus_type      star_bus_type_t;
6
7  /* Callback function types also use _t suffix */
8  typedef esp_err_t (*star_bus_callback_t)(star_bus_config_t* bus, void* user_ctx);
9  typedef esp_err_t (*pin_registration_function_t)(gpio_num_t pin_num,
10                                                  const char* bus_name,
11                                                  const char* pin_usage,
12                                                  void* user_ctx);
13
14  /* WRONG */
15  typedef struct StarBusConfig StarBusConfig; /* PascalCase */
16  typedef struct star_bus_config star_bus_config; /* missing _t suffix */
```

18.6 Enum Members

Enum members use the k_ prefix followed by snake_case.

```
1  /* CORRECT - k_prefix with snake_case */
2  typedef enum {
3      k_star_bus_type_none,    /**< No bus type specified */
4      k_star_bus_type_i2c,    /**< I2C bus */
5      k_star_bus_type_spi,    /**< SPI bus */
6      k_star_bus_type_gpio,   /**< GPIO bus */
7      k_star_bus_type_adc,    /**< ADC bus */
8      k_star_bus_type_count,  /**< Number of bus types (sentinel) */
9  } star_bus_type_t;
10
11  /* Usage in switch statements */
12  switch (config->type) {
13      case k_star_bus_type_i2c: {
14          /* Handle I2C */
15          break;
```

```

16     }
17     case k_star_bus_type_spi: {
18         /* Handle SPI */
19         break;
20     }
21     default: {
22         ESP_LOGW(s_TAG, "Unknown bus type: %d", config->type);
23         break;
24     }
25 }
26
27 /* WRONG */
28 typedef enum {
29     STAR_BUS_TYPE_I2C,      /* SCREAMING_SNAKE_CASE without k_ */
30     StarBusTypeI2c,        /* PascalCase */
31     starBusTypeI2c,        /* camelCase */
32 } bad_enum_t;

```

18.7 Static Variables (File-Scoped)

Static file-scoped variables use the `s_` prefix followed by `snake_case`.

```

1  /* CORRECT - s_ prefix for static file-scoped variables */
2  static const char* s_TAG = "STAR_BUS_MANAGER";
3  static const uint32_t s_i2c_timeout_ms = 1000;
4
5  /* From star_bus_i2c.c */
6  static const char* s_TAG = "STAR_BUS_I2C";
7  static const uint32_t s_i2c_timeout_ms = 1000;
8
9  /* From star_bus_config.c */
10 static const char* s_TAG = "STAR_BUS_CONFIG";
11
12 /* WRONG - missing s_ prefix */
13 static const char* TAG = "MODULE";
14 static uint32_t timeout_ms = 1000;

```

18.8 Global Variables

Global variables use the `g_` prefix followed by `snake_case`. However, global variables are **strongly discouraged**. Prefer dependency injection or static file-scoped variables.

```
1  /* CORRECT (but discouraged) - g_ prefix for globals */
2  g_system_state_t g_system_state;
3  uint32_t g_error_count;
4
5  /* PREFERRED - use dependency injection instead */
6  typedef struct {
7      star_error_interface_t* error_iface; /* Injected */
8      star_pin_interface_t*  pin_iface;   /* Injected */
9  } star_bus_manager_t;
10
11 /* Initialize with injected dependencies */
12 esp_err_t star_bus_manager_init(star_bus_manager_t* manager,
13                                const char*          tag,
14                                star_error_interface_t* error_iface,
15                                star_pin_interface_t*  pin_iface);
```

18.9 Private Functions

18.9.1 File-Scoped Static Functions (`internal_` prefix)

Static functions that are internal to a single source file use the `internal_` prefix.

```
1  /* From star_bus_manager.c - internal helper functions */
2  static esp_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
3                                             gpio_num_t           pin,
4                                             const char*          bus_name,
5                                             const char*          usage,
6                                             bool                 can_be_shared)
7  {
8      if (pin < 0 || pin >= GPIO_NUM_MAX) {
9          return ESP_OK; /* Not an error, pin is not used */
10     }
11     /* ... implementation ... */
12 }
13
```

```

14 static esp_err_t internal_unregister_bus_pin(star_pin_interface_t* pin_iface,
15                                             gpio_num_t          pin,
16                                             const char*        bus_name,
17                                             const char*        usage)
18 {
19     /* ... implementation ... */
20 }
21
22 static esp_err_t internal_register_config_pins(star_pin_interface_t* pin_iface,
23                                              const star_bus_config_t* config)
24 {
25     /* ... implementation ... */
26 }
27
28 /* From star_bus_i2c.c */
29 static esp_err_t internal_star_bus_i2c_write(const star_bus_config_t* config,
30                                             const uint8_t*          data,
31                                             size_t                  len,
32                                             uint8_t                  reg_addr,
33                                             size_t*                  bytes_written);
34
35 static esp_err_t internal_i2c_execute_with_retry(const star_bus_config_t* config,
36                                                  i2c_port_t              port,
37                                                  i2c_cmd_handle_t         cmd,
38                                                  const char*              operation_name);

```

18.9.2 Exposed Private Functions (priv_ prefix)

Functions that must be exposed across files within a module (but are not part of the public API) use the `priv_` prefix.

```

1 /* In private header (e.g., star_bus_internal.h) */
2 /* These functions are used by multiple .c files in the same module
3    but should not be called by external code */
4
5 esp_err_t priv_star_bus_validate_config(const star_bus_config_t* config);
6 esp_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);

```

```

7 void      priv_star_bus_release_mutex(star_bus_manager_t* manager);
8
9 /* Used in implementation files within the module */
10 esp_err_t star_bus_config_init(star_bus_config_t* config,
11                               struct star_bus_manager* manager)
12 {
13     /* Validate using private helper */
14     esp_err_t ret = priv_star_bus_validate_config(config);
15     if (ret != ESP_OK) {
16         return ret;
17     }
18     /* ... rest of implementation ... */
19 }

```

18.10 Summary Table

Identifier Type	Convention	Example
Functions	snake_case	star_bus_manager_init
Variables	snake_case	bytes_written
Constants/Macros	SCREAMING_SNAKE_CASE	STAR_BUS_NAME_MAX_LEN
Type names	snake_case_t	star_bus_config_t
Enum members	k_snake_case	k_star_bus_type_i2c
Static variables	s_snake_case	s_TAG
Global variables	g_snake_case	g_system_state
Internal functions	internal_snake_case	internal_register_bus_pin
Private functions	priv_snake_case	priv_star_bus_validate_config
Module prefix	star_[module]_	star_bus_, star_error_

19.0 Type Definitions

This section describes the patterns and conventions for defining custom types in the STAR firmware codebase. Consistent type definition patterns improve code readability and enable better tooling support.

19.1 Typedef Struct Pattern

All structures should be defined using the typedef pattern with the `_t` suffix.

19.1.1 Standard Pattern

```
1  /* Forward declaration (in header for dependencies) */
2  typedef struct star_bus_config star_bus_config_t;
3  typedef struct star_bus_manager star_bus_manager_t;
4
5  /* Full definition (in header or implementation) */
6  typedef struct star_bus_config {
7      const char*      name;          /**< Unique name for this bus/device instance */
8      star_bus_type_t type;          /**< Type of bus (I2C, SPI, etc.) */
9      bool             initialized; /**< Whether this bus has been initialized */
10     void*             handle;       /**< Driver/device handle */
11     void*             user_ctx;     /**< User context pointer for callbacks */
12
13     /* Protocol-specific configuration (see union section) */
14     union {
15         star_i2c_bus_config_t i2c;
16         star_spi_bus_config_t spi;
17         star_gpio_bus_config_t gpio;
18         star_adc_bus_config_t adc;
19     } proto;
20
21     struct star_bus_config* next; /**< Linked list pointer */
22 } star_bus_config_t;
```

19.1.2 Error Handler Example

```
1  /* From star_error_handler.h */
2  typedef struct error_handler {
3      uint32_t      error_count;      /**< Total number of errors recorded */
4      uint32_t      max_retries;      /**< Maximum retry attempts allowed */
5      uint32_t      current_retry;    /**< Current retry attempt counter */
6      uint32_t      base_retry_delay; /**< Base delay between retries (ms) */
7      uint32_t      max_retry_delay;  /**< Maximum retry delay (ms) */
8      uint32_t      current_retry_delay; /**< Current delay with backoff */
9      esp_err_t     last_error;       /**< Last recorded error code */
10     bool           in_error_state;   /**< Whether in error state */
11     void*          reset_context;    /**< Context for reset function */
12 }
```

```

12 SemaphoreHandle_t mutex;                /**< Mutex for thread-safety */
13
14 esp_err_t (*reset_fn)(void* context); /**< Optional reset function */
15 } error_handler_t;

```

19.2 Union Usage for Protocol Configurations

Unions are used to store protocol-specific configurations efficiently. This pattern allows a single configuration structure to support multiple bus types.

```

1  /* From star_bus_manager_types.h */
2  typedef struct star_bus_config {
3      const char*    name;
4      star_bus_type_t type; /* Discriminator for the union */
5      bool           initialized;
6      void*          handle;
7      void*          user_ctx;
8
9      /* Union holds protocol-specific configuration */
10     union {
11         star_i2c_bus_config_t i2c; /**< I2C-specific configuration */
12         star_spi_bus_config_t spi; /**< SPI-specific configuration */
13         star_gpio_bus_config_t gpio; /**< GPIO-specific configuration */
14         star_adc_bus_config_t adc; /**< ADC-specific configuration */
15     } proto;
16
17     struct star_bus_config* next;
18 } star_bus_config_t;
19
20 /* Accessing union members based on type discriminator */
21 switch (config->type) {
22     case k_star_bus_type_i2c: {
23         i2c_port_t port = config->proto.i2c.port;
24         uint8_t address = config->proto.i2c.address;
25         /* ... */
26         break;
27     }
28     case k_star_bus_type_spi: {
29         spi_host_device_t host = config->proto.spi.host;

```

```

30     gpio_num_t copi_pin = config->proto.spi.copi_io_num;
31     /* ... */
32     break;
33 }
34 /* ... other cases ... */
35 }

```

19.2.1 Protocol-Specific Structures

```

1  /* I2C bus configuration (from star_bus_protocol_types.h) */
2  typedef struct star_i2c_bus_config {
3      i2c_port_t      port;          /**< I2C port number */
4      i2c_config_t    config;        /**< ESP-IDF I2C configuration */
5      uint8_t         address;       /**< 7-bit I2C device address */
6      star_i2c_callbacks_t callbacks; /**< User callbacks */
7      star_i2c_ops_t   ops;          /**< Operation function pointers */
8  } star_i2c_bus_config_t;
9
10 /* SPI bus configuration */
11 typedef struct star_spi_bus_config {
12     spi_host_device_t host;          /**< SPI host number */
13     bool               is_peripheral; /**< True if SPI peripheral mode */
14     spi_bus_config_t   bus_cfg;       /**< ESP-IDF SPI bus config */
15     spi_device_interface_config_t dev_cfg; /**< Device config */
16     star_spi_callbacks_t callbacks;
17     star_spi_ops_t     ops;
18
19     /* Inclusive terminology pin names (see Terminology section) */
20     gpio_num_t copi_io_num; /**< Controller Out, Peripheral In */
21     gpio_num_t cipo_io_num; /**< Controller In, Peripheral Out */
22     gpio_num_t sclk_io_num; /**< Serial Clock */
23     gpio_num_t cs_io_num;    /**< Chip Select */
24     /* ... */
25 } star_spi_bus_config_t;

```

19.3 Callback Function Types

Callback function pointer types follow a consistent pattern with the `_t` suffix.

```

1  /* From star_bus_common_types.h - Pin registration callback */
2  typedef esp_err_t (*pin_registration_function_t)(gpio_num_t pin_num,
3                                                    const char* bus_name,
4                                                    const char* pin_usage,
5                                                    void* user_ctx);
6
7  /* From star_bus_manager.h - Bus access callback */
8  typedef esp_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
9                                           void* user_ctx);
10
11 /* From star_bus_function_types.h - I2C operation callbacks */
12 typedef esp_err_t (*star_i2c_write_fn_t)(const star_bus_config_t* config,
13                                          const uint8_t* data,
14                                          size_t len,
15                                          uint8_t reg_addr,
16                                          size_t* bytes_written);
17
18 typedef esp_err_t (*star_i2c_read_fn_t)(const star_bus_config_t* config,
19                                         uint8_t* data,
20                                         size_t len,
21                                         uint8_t reg_addr,
22                                         size_t* bytes_read);
23
24 /* Event callbacks */
25 typedef void (*star_i2c_transfer_complete_cb_t)(const star_bus_event_t* event,
26                                                 void* user_ctx);
27
28 typedef void (*star_i2c_error_cb_t)(const star_bus_event_t* event,
29                                    esp_err_t error,
30                                    void* user_ctx);
31
32 /* Grouping related callbacks in a struct */
33 typedef struct star_i2c_callbacks {
34     star_i2c_transfer_complete_cb_t on_transfer_complete;
35     star_i2c_error_cb_t on_error;
36 } star_i2c_callbacks_t;

```

19.4 Interface Patterns (Dependency Injection)

Interfaces use structs with function pointers and an opaque context pointer. This pattern enables dependency injection and testability.

```
1  /* From star_error_interface.h */
2  typedef struct star_error_interface {
3      /**
4       * @brief Record an error
5       * @param ctx Implementation context
6       * @param error Error code
7       * @param message Error message
8       * @param file Source file
9       * @param line Line number
10      * @param func Function name
11      * @return ESP_OK on success
12      */
13      esp_err_t (*record_error)(void*      ctx,
14                               esp_err_t  error,
15                               const char* message,
16                               const char* file,
17                               int         line,
18                               const char* func);
19
20      /**
21       * @brief Check if retry is possible
22       * @param ctx Implementation context
23       * @return true if retry is allowed
24       */
25      bool (*can_retry)(void* ctx);
26
27      /**
28       * @brief Reset error state
29       * @param ctx Implementation context
30       * @return ESP_OK on success
31       */
32      esp_err_t (*reset_state)(void* ctx);
33
34      /**
```

```

35     * @brief Implementation context (opaque pointer)
36     */
37     void* ctx;
38 } star_error_interface_t;
39
40 /* From star_pin_interface.h */
41 typedef struct star_pin_interface {
42     esp_err_t (*register_pin)(void* ctx, int pin,
43                               const char* description, bool shared);
44     esp_err_t (*unregister_pin)(void* ctx, int pin,
45                                const char* description);
46     esp_err_t (*validate)(void* ctx);
47     void* ctx;
48 } star_pin_interface_t;

```

19.4.1 Interface Usage Pattern

```

1  /* Creating an interface from concrete implementation */
2  error_handler_t error_handler;
3  error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
4
5  star_error_interface_t error_iface;
6  error_handler_get_interface(&error_iface, &error_handler);
7
8  /* Injecting interfaces into components */
9  star_bus_manager_t bus_manager;
10 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
11
12 /* Using interface through function pointers */
13 if (error_iface.can_retry(error_iface.ctx)) {
14     /* Retry operation */
15 }
16
17 /* Or using the convenience macro */
18 STAR_IFACE_RECORD_ERROR(&error_iface, ESP_ERR_TIMEOUT, "Operation timed out");

```

19.5 Opaque Handle Patterns

For complex objects that should not be directly manipulated, use opaque handles.

```
1  /* Forward declaration creates an opaque type */
2  typedef struct star_bus_manager star_bus_manager_t;
3
4  /* Public API only uses pointers to the opaque type */
5  esp_err_t star_bus_manager_init(star_bus_manager_t* manager, ...);
6  esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager, ...);
7
8  /* Full definition is only in the .c file or private header */
9  /* Users cannot access internal fields directly */
10
11 /* Example usage - users allocate storage but don't access internals */
12 star_bus_manager_t bus_manager; /* Stack allocation */
13 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
14
15 /* Or heap allocation */
16 star_bus_manager_t* manager_ptr = malloc(sizeof(star_bus_manager_t));
17 star_bus_manager_init(manager_ptr, "main", &error_iface, &pin_iface);
```

19.6 Operations Structure Pattern

Group related function pointers in an operations structure for pluggable implementations.

```
1  /* From star_bus_protocol_types.h */
2  typedef struct star_i2c_ops {
3      star_i2c_write_fn_t      write;          /**< Write with register */
4      star_i2c_read_fn_t       read;           /**< Read with register */
5      star_i2c_write_command_fn_t write_command; /**< Write command byte */
6      star_i2c_read_raw_fn_t    read_raw;       /**< Read raw bytes */
7  } star_i2c_ops_t;
8
9  typedef struct star_spi_ops {
10     star_spi_transmit_fn_t transmit; /**< Transmit only */
11     star_spi_receive_fn_t receive; /**< Receive only */
12     star_spi_transfer_fn_t transfer; /**< Full-duplex transfer */
13 } star_spi_ops_t;
14
```

```

15 typedef struct star_gpio_ops {
16     esp_err_t (*write_digital)(const struct star_bus_config* config,
17                               uint8_t pin_index, uint8_t value);
18     esp_err_t (*read_digital)(const struct star_bus_config* config,
19                               uint8_t pin_index, uint8_t* value);
20 } star_gpio_ops_t;
21
22 /* Initializing default operations */
23 void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
24 {
25     if (ops == NULL) {
26         ESP_LOGE(s_TAG, "Cannot initialize NULL ops pointer");
27         return;
28     }
29     ops->write      = internal_star_bus_i2c_write;
30     ops->read       = internal_star_bus_i2c_read;
31     ops->write_command = internal_star_bus_i2c_write_command;
32     ops->read_raw   = internal_star_bus_i2c_read_raw;
33 }

```

19.7 Manager Structure Pattern

Manager structures coordinate multiple resources with thread-safe access.

```

1  /* From star_bus_manager_types.h */
2  typedef struct star_bus_manager {
3      star_bus_config_t*    buses;          /**< Linked list of bus configs */
4      SemaphoreHandle_t     mutex;          /**< Mutex for thread-safety */
5      const char*           tag;            /**< Logging tag */
6      star_error_interface_t* error_iface;  /**< Injected error handler */
7      star_pin_interface_t* pin_iface;      /**< Injected pin validator */
8
9      /* Resource tracking */
10     bool spi_host_initialized[SPI_HOST_MAX];
11     int  spi_device_count[SPI_HOST_MAX];
12 } star_bus_manager_t;

```

19.8 Best Practices

- **Always use `_t` suffix:** Makes types easily distinguishable from variables.
- **Use forward declarations:** Minimize header dependencies and compilation time.
- **Document all fields:** Use Doxygen comments for struct members.
- **Keep unions discriminated:** Always pair unions with a type field.
- **Prefer static const over `#define`:** For typed constants within structures.
- **Use opaque types for encapsulation:** Hide implementation details from users.
- **Group related function pointers:** Use ops structs for pluggable implementations.

20.0 File Organization

This section describes the standard file and directory structure for the STAR firmware codebase. Consistent organization improves navigability and maintainability.

20.1 Library Directory Structure

Each library follows a standard PlatformIO/ESP-IDF component structure:

```
lib/star_component/  
    CMakeLists.txt          # CMake build configuration  
    library.json            # PlatformIO library metadata  
    include/                # Public headers (API)  
        star_component.h  
        star_component_types.h  
    src/                    # Implementation files  
        star_component.c
```

20.1.1 Example: star_bus Library

```
lib/star_bus/  
    CMakeLists.txt  
    library.json  
    include/  
        star_bus_adc.h  
        star_bus_common_types.h  
        star_bus_config.h  
        star_bus_event_types.h
```

```

    star_bus_function_types.h
    star_bus_gpio.h
    star_bus_i2c.h
    star_bus_manager.h
    star_bus_manager_types.h
    star_bus_protocol_types.h
    star_bus_spi.h
    star_bus_types.h
src/
    star_bus_adc.c
    star_bus_config.c
    star_bus_gpio.c
    star_bus_i2c.c
    star_bus_manager.c
    star_bus_spi.c
    star_bus_types.c

```

20.2 CMakeLists.txt Structure

```

1 # lib/star_bus/CMakeLists.txt
2
3 idf_component_register(
4     SRCS
5         "src/star_bus_adc.c"
6         "src/star_bus_config.c"
7         "src/star_bus_gpio.c"
8         "src/star_bus_i2c.c"
9         "src/star_bus_manager.c"
10        "src/star_bus_spi.c"
11        "src/star_bus_types.c"
12    INCLUDE_DIRS
13        "include"
14    REQUIRES
15        driver
16        esp_adc
17        freertos
18        star_core

```

20.3 Header File Organization

Header files must follow a consistent internal structure.

20.3.1 Header File Template

```

1  /* lib/star_bus/include/star_bus_manager.h */
2
3  #ifndef STAR_COMPONENT_BUS_MANAGER_H
4  #define STAR_COMPONENT_BUS_MANAGER_H
5
6  #ifdef __cplusplus
7  extern "C" {
8  #endif
9
10 /* === Includes === */
11 #include "esp_err.h"
12 #include "star_bus_manager_types.h"
13
14 /**
15  * @file star_bus_manager.h
16  * @brief Unified bus manager for I2C, SPI, GPIO, and other bus protocols
17  *
18  * The bus manager provides a central registry for managing multiple
19  * bus configurations. It supports dependency injection of error handlers
20  * and pin validators for loose coupling.
21  *
22  * Key Features:
23  * - Thread-safe bus management with mutex protection
24  * - Support for multiple bus types (I2C, SPI, GPIO, etc.)
25  * - Automatic pin registration and conflict detection
26  * - Safe bus access via callback pattern
27  *
28  * Example Usage:
29  * @code
30  * // Initialize bus manager with dependency injection

```

```

31 * star_bus_manager_t bus_manager;
32 * star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
33 *
34 * // Create and add an I2C bus
35 * star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
36 *     "sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22, 400000);
37 * star_bus_manager_add_bus(&bus_manager, i2c_bus);
38 * @endcode
39 */
40
41 /* === Forward Declarations === */
42 /* (if needed beyond what's in included headers) */
43
44 /* === Public Functions === */
45
46 /**
47 * @brief Initialize a bus manager instance.
48 *
49 * @param[out] manager    Pointer to bus manager to initialize. Must not be NULL.
50 * @param[in]  tag        Optional tag string for logging. NULL uses default.
51 * @param[in]  error_iface Optional error handler interface (can be NULL).
52 * @param[in]  pin_iface  Optional pin validator interface (can be NULL).
53 * @return esp_err_t ESP_OK on success, ESP_ERR_INVALID_ARG if manager is NULL,
54 *         ESP_ERR_NO_MEM if mutex creation fails.
55 */
56 esp_err_t star_bus_manager_init(star_bus_manager_t*    manager,
57                                const char*            tag,
58                                star_error_interface_t* error_iface,
59                                star_pin_interface_t*   pin_iface);
60
61 /* ... more function declarations ... */
62
63 #ifndef __cplusplus
64 }
65 #endif
66
67 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

20.3.2 Header Section Order

1. **File path comment:** Identifies the file location
2. **Include guard:** `#ifndef STAR_COMPONENT_FILENAME_H`
3. **C++ extern “C” opening:** For C++ compatibility
4. **Includes:** System, then ESP-IDF, then project headers
5. **File documentation:** `@file` Doxygen block with overview
6. **Forward declarations:** If needed
7. **Constants and macros:** Public defines
8. **Type definitions:** Enums, structs, typedefs
9. **Function declarations:** Grouped by category
10. **C++ extern “C” closing:** Matches opening
11. **Include guard closing:** `#endif /* ... */`

20.4 Include Guard Naming Convention

Include guards follow the pattern: `STAR_COMPONENT_FILENAME_H`

```
1  /* star_bus_manager.h */
2  #ifndef STAR_COMPONENT_BUS_MANAGER_H
3  #define STAR_COMPONENT_BUS_MANAGER_H
4  /* ... */
5  #endif /* STAR_COMPONENT_BUS_MANAGER_H */
6
7  /* star_bus_config.h */
8  #ifndef STAR_COMPONENT_BUS_CONFIG_H
9  #define STAR_COMPONENT_BUS_CONFIG_H
10 /* ... */
11 #endif /* STAR_COMPONENT_BUS_CONFIG_H */
12
13 /* star_error_interface.h */
14 #ifndef STAR_ERROR_INTERFACE_H
15 #define STAR_ERROR_INTERFACE_H
16 /* ... */
17 #endif /* STAR_ERROR_INTERFACE_H */
18
19 /* star_pin_interface.h */
20 #ifndef STAR_PIN_INTERFACE_H
21 #define STAR_PIN_INTERFACE_H
```

```

22  /* ... */
23  #endif /* STAR_PIN_INTERFACE_H */

```

20.5 Source File Organization

Source files follow a consistent internal structure.

20.5.1 Source File Template

```

1  /* lib/star_bus/src/star_bus_manager.c */
2
3  /* === Primary Header === */
4  #include "star_bus_manager.h"
5
6  /* === System Includes === */
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/semphr.h"
9
10 #include <stdio.h>
11 #include <stdlib.h>
12 #include <string.h>
13
14 /* === ESP-IDF Includes === */
15 #include "esp_check.h"
16 #include "esp_log.h"
17 #include "sdkconfig.h"
18
19 /* === Project Includes === */
20 #include "star_bus_adc.h"
21 #include "star_bus_config.h"
22 #include "star_bus_gpio.h"
23
24 /* === Constants === */
25
26 static const char* s_TAG = "STAR_BUS_MANAGER";
27
28 /* === Private Function Prototypes === */
29

```

```

30 static esp_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
31                                           gpio_num_t          pin,
32                                           const char*         bus_name,
33                                           const char*         usage,
34                                           bool                 can_be_shared);
35
36 static esp_err_t internal_unregister_bus_pin(star_pin_interface_t* pin_iface,
37                                              gpio_num_t          pin,
38                                              const char*         bus_name,
39                                              const char*         usage);
40
41 /* === Public Functions === */
42
43 esp_err_t star_bus_manager_init(star_bus_manager_t* manager,
44                                const char*         tag,
45                                star_error_interface_t* error_iface,
46                                star_pin_interface_t* pin_iface)
47 {
48     ESP_RETURN_ON_FALSE(manager, ESP_ERR_INVALID_ARG, s_TAG,
49                         "Manager pointer is NULL");
50
51     manager->buses      = NULL;
52     manager->error_iface = error_iface;
53     manager->pin_iface  = pin_iface;
54
55     /* ... rest of implementation ... */
56
57     return ESP_OK;
58 }
59
60 /* ... more public functions ... */
61
62 /* === Private Functions === */
63
64 static esp_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
65                                           gpio_num_t          pin,
66                                           const char*         bus_name,
67                                           const char*         usage,

```

```

68         bool                                can_be_shared)
69 {
70     if (pin < 0 || pin >= GPIO_NUM_MAX) {
71         return ESP_OK; /* Not an error, pin is not used */
72     }
73     /* ... implementation ... */
74 }

```

20.5.2 Source Section Order

1. **File path comment:** Identifies the file location
2. **Primary header:** The corresponding .h file for this .c file
3. **System includes:** FreeRTOS, standard library (<...>)
4. **ESP-IDF includes:** ESP-IDF components
5. **Project includes:** Other project headers ("...")
6. **Constants:** static const variables, including s_TAG
7. **Private function prototypes:** Forward declarations of static functions
8. **Public functions:** Implementation of API functions
9. **Private functions:** Implementation of static helper functions

20.6 Types Header Separation

For complex modules, separate type definitions into dedicated headers:

```

star_bus/include/
    star_bus_manager.h      # Main API (functions)
    star_bus_manager_types.h # Types for manager
    star_bus_common_types.h # Shared types (enums, forward decls)
    star_bus_protocol_types.h # Protocol-specific types
    star_bus_function_types.h # Function pointer types
    star_bus_event_types.h  # Event structures
    star_bus_types.h        # Controller header (includes all)

```

20.6.1 Controller Header Pattern

```

1  /* star_bus_types.h - includes all type headers */
2  #ifndef STAR_COMPONENT_BUS_TYPES_H
3  #define STAR_COMPONENT_BUS_TYPES_H

```

```

4
5 #ifndef __cplusplus
6 extern "C" {
7 #endif
8
9 /* Controller include file for all public types */
10 #include "star_bus_common_types.h"
11 #include "star_bus_event_types.h"
12 #include "star_bus_function_types.h"
13 #include "star_bus_manager_types.h"
14 #include "star_bus_protocol_types.h"
15
16 #ifndef __cplusplus
17 }
18 #endif
19
20 #endif /* STAR_COMPONENT_BUS_TYPES_H */

```

20.7 Project Root Structure

```

star-esp32-firmware/
  CLAUDE.md          # Claude Code instructions
  platformio.ini      # PlatformIO configuration
  sdkconfig.defaults  # ESP-IDF default config
  lib/               # Libraries/components
    star_core/        # Core interfaces
    star_bus/         # Bus abstraction
    star_error_handler/
    star_pin_validator/
    star_sensor_*/    # Sensor drivers
    star_motor/
    star_pid/
  src/               # Main application
    main.c
  partitions/        # Partition tables
    partitions_4mb.csv
    partitions_16mb.csv

```

```
docs/                # Documentation
    style_guide.tex
    style_guide_sections/
scripts/             # Build/utility scripts
test/                # Test files
```

20.8 Best Practices

- **One header per component:** Each component has its primary public header.
- **Minimize header dependencies:** Use forward declarations to reduce includes.
- **Self-contained headers:** Each header should compile independently.
- **Consistent naming:** Files match the component they contain.
- **Group related files:** Keep related headers and sources together.
- **Separate types from functions:** For complex modules, split into `_types.h`.
- **Document file purpose:** Use `@file` Doxygen blocks.

21.0 Inclusive Terminology

This section establishes the OSHWA (Open Source Hardware Association) inclusive terminology standards used throughout the STAR firmware codebase. These standards replace legacy terminology with more inclusive alternatives.

21.1 Overview

The STAR project follows inclusive terminology guidelines to ensure our codebase is welcoming to all contributors and avoids language with harmful historical connotations. This is particularly relevant in embedded systems where certain legacy terms have been deeply entrenched.

21.2 SPI Bus Terminology

21.2.1 COPI/CIPO (Not MOSI/MISO)

STAR Term	Legacy Term	Description
COPI	MOSI	Controller Out, Peripheral In
CIPO	MISO	Controller In, Peripheral Out
SCLK	SCLK	Serial Clock (unchanged)
CS	SS	Chip Select (unchanged)

```

1  /* CORRECT - OSHWA terminology */
2  typedef struct star_spi_bus_config {
3      gpio_num_t copi_io_num;  /**< Controller Out, Peripheral In */
4      gpio_num_t cipo_io_num;  /**< Controller In, Peripheral Out */
5      gpio_num_t sclk_io_num;  /**< Serial Clock */
6      gpio_num_t cs_io_num;    /**< Chip Select */
7      /* ... */
8  } star_spi_bus_config_t;
9
10 /* Factory function uses inclusive naming */
11 star_bus_config_t* star_bus_config_create_spi_device(
12     const char*      name,
13     spi_host_device_t host,
14     gpio_num_t       copi_pin,  /* NOT mosi_pin */
15     gpio_num_t       cipo_pin,  /* NOT miso_pin */
16     gpio_num_t       sclk_pin,
17     int32_t          dma_chan,
18     const spi_device_interface_config_t* dev_cfg);
19
20 /* In documentation and comments */
21 /* CORRECT */
22 reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
23                                     bus_name,
24                                     "SPI COPI", /* Descriptive usage */
25                                     reg_func,
26                                     user_ctx,
27                                     true);
28
29 /* WRONG */
30 reg_result = register_pin_if_valid(curr->proto.spi.mosi_io_num,
31                                     bus_name,
32                                     "SPI MOSI", /* Legacy terminology */
33                                     reg_func,
34                                     user_ctx,
35                                     true);

```

21.3 I2C and General Bus Terminology

21.3.1 Controller/Peripheral (Not Master/Slave)

STAR Term	Legacy Term	Description
Controller	Master	Device that initiates communication
Peripheral	Slave	Device that responds to controller
Primary	Master	For configuration structures
Main	Master	For general reference

```
1  /* CORRECT - Controller/Peripheral terminology */
2  typedef struct star_spi_bus_config {
3      spi_host_device_t host;
4      bool is_peripheral;  /**< True if operating as SPI peripheral */
5      /* ... */
6  } star_spi_bus_config_t;
7
8  /* Factory function for peripheral mode */
9  star_bus_config_t* star_bus_config_create_spi_peripheral(
10     const char*      name,
11     spi_host_device_t host,
12     gpio_num_t       copi_pin,
13     gpio_num_t       cipo_pin,
14     gpio_num_t       sclk_pin,
15     gpio_num_t       cs_pin,
16     int32_t          queue_size,
17     uint8_t          mode);
18
19  /* In comments and documentation */
20  /**
21   * @brief I2C Controller operations
22   *
23   * The controller device initiates all transactions on the bus.
24   * Peripheral devices respond when addressed by the controller.
25   */
26
27  /* WRONG */
28  bool is_slave;          /* Use is_peripheral */
29  star_bus_config_create_spi_slave(); /* Use _peripheral */
```

21.4 Mapping ESP-IDF Legacy APIs

ESP-IDF still uses legacy terminology internally. When interfacing with ESP-IDF, document the mapping clearly.

```
1  /* From star_bus_protocol_types.h */
2  typedef struct star_spi_bus_config {
3      /* ... */
4
5      /* SPI pin naming follows OSHWA standard:
6       * - COPI (Controller Out, Peripheral In) - replaces MOSI
7       * - CIPO (Controller In, Peripheral Out) - replaces MISO
8       * This modern terminology removes master/slave language.
9       * The ESP-IDF still uses mosi_io_num/miso_io_num internally,
10      * which we map here.
11      */
12      gpio_num_t copi_io_num; /**< GPIO for COPI (maps to ESP-IDF mosi_io_num) */
13      gpio_num_t cipo_io_num; /**< GPIO for CIPO (maps to ESP-IDF miso_io_num) */
14      /* ... */
15  } star_spi_bus_config_t;
16
17  /* When setting up ESP-IDF structures */
18  static esp_err_t internal_setup_spi_bus(star_bus_config_t* config)
19  {
20      spi_bus_config_t bus_cfg = {
21          /* Map our inclusive terminology to ESP-IDF legacy names */
22          .mosi_io_num    = config->proto.spi.copi_io_num,
23          .miso_io_num    = config->proto.spi.cipo_io_num,
24          .sclk_io_num    = config->proto.spi.sclk_io_num,
25          .quadwp_io_num  = -1,
26          .quadhd_io_num  = -1,
27          .max_transfer_sz = config->proto.spi.max_transfer_sz,
28      };
29      /* ... */
30  }
31
32  /* Similarly for I2C */
33  /* ESP-IDF uses I2C_MODE_SLAVE, but we document as peripheral mode */
34  i2c_config_t i2c_conf = {
```

```

35     .mode = I2C_MODE_MASTER,  /* Controller mode in STAR terminology */
36     /* or */
37     .mode = I2C_MODE_SLAVE,   /* Peripheral mode in STAR terminology */
38     /* ... */
39 };

```

21.5 Documentation Guidelines

When writing comments, documentation, or user-facing text:

```

1  /* CORRECT - Use inclusive terminology in all documentation */
2
3  /**
4   * @brief Configure ESP32 as SPI peripheral device
5   *
6   * The peripheral will respond to transactions initiated by an
7   * external SPI controller. Data flows:
8   * - COPI: Data received from controller
9   * - CIP0: Data sent to controller
10  *
11  * @param[in] name Unique name for this SPI peripheral instance
12  * @param[in] host SPI host device
13  * @param[in] copi_pin GPIO for Controller Out, Peripheral In
14  * @param[in] cipo_pin GPIO for Controller In, Peripheral Out
15  * ...
16  */
17 star_bus_config_t* star_bus_config_create_spi_peripheral(...);
18
19 /* WRONG - Legacy terminology */
20 /**
21  * @brief Configure ESP32 as SPI slave device
22  *
23  * The slave will respond to transactions initiated by the
24  * SPI master. Data flows:
25  * - MOSI: Data from master to slave
26  * - MISO: Data from slave to master
27  */

```

21.6 Code Review Checklist

When reviewing code, check for:

- **Variable names:** No “master”, “slave”, “mosi”, “miso”
- **Function names:** Use “controller”, “peripheral”, “copi”, “cipo”
- **Comments:** Update any legacy terminology
- **Documentation:** Ensure Doxygen uses inclusive terms
- **String literals:** Check log messages and error strings
- **ESP-IDF mapping:** Document when interfacing with legacy APIs

21.7 Common Patterns

```
1  /* Logging with inclusive terminology */
2  ESP_LOGI(s_TAG, "SPI peripheral initialized on host %d", host);
3  ESP_LOGI(s_TAG, "I2C controller configured on port %d", port);
4
5  /* Error messages */
6  ESP_LOGE(s_TAG, "Failed to initialize SPI peripheral: %s",
7           esp_err_to_name(ret));
8  ESP_LOGE(s_TAG, "I2C controller transaction failed");
9
10 /* Pin registration descriptions */
11 internal_register_bus_pin(pin_iface, config->proto.spi.copi_io_num,
12                           config->name, "SPI COPI", true);
13 internal_register_bus_pin(pin_iface, config->proto.spi.cipo_io_num,
14                           config->name, "SPI CIPO", true);
```

21.8 Summary

Use This	Not This
Controller	Master
Peripheral	Slave
COPI	MOSI
CIFO	MISO
Primary	Master (for configs)
Main	Master (general)
Allowlist	Whitelist
Blocklist	Blacklist

Following these guidelines ensures our codebase is inclusive and welcoming while maintaining technical clarity. The terminology accurately describes the functional roles of devices without relying on harmful historical language.

22.0 Architecture Patterns

This section describes the key architectural patterns used throughout the STAR firmware codebase. These patterns enable loose coupling, testability, and maintainability in embedded systems.

22.1 Dependency Injection (DIP)

The Dependency Inversion Principle (DIP) is the foundational architectural pattern of the STAR framework. High-level modules depend on abstract interfaces rather than concrete implementations.

22.1.1 Interface Definition

Interfaces are defined as structs containing function pointers and an opaque context pointer.

```
1 /* From star_error_interface.h */
2 typedef struct star_error_interface {
3     esp_err_t (*record_error)(void* ctx, esp_err_t error,
4                               const char* message, const char* file,
5                               int line, const char* func);
6     bool (*can_retry)(void* ctx);
7     esp_err_t (*reset_state)(void* ctx);
8     void* ctx; /* Opaque context pointer */
```

```

9 } star_error_interface_t;
10
11 /* From star_pin_interface.h */
12 typedef struct star_pin_interface {
13     esp_err_t (*register_pin)(void* ctx, int pin,
14                               const char* description, bool shared);
15     esp_err_t (*unregister_pin)(void* ctx, int pin,
16                                const char* description);
17     esp_err_t (*validate)(void* ctx);
18     void* ctx;
19 } star_pin_interface_t;

```

22.1.2 Interface Implementation

Concrete implementations provide the actual functionality.

```

1 /* Create error handler and get interface */
2 error_handler_t error_handler;
3 error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
4
5 star_error_interface_t error_iface;
6 error_handler_get_interface(&error_iface, &error_handler);
7
8 /* Implementation of get_interface */
9 void error_handler_get_interface(star_error_interface_t* iface,
10                                error_handler_t* handler)
11 {
12     iface->record_error = adapter_record_error; /* Adapter function */
13     iface->can_retry     = adapter_can_retry;
14     iface->reset_state   = adapter_reset_state;
15     iface->ctx           = handler; /* Concrete handler as context */
16 }

```

22.1.3 Dependency Injection Pattern

Components receive their dependencies through initialization functions.

```

1 /* Component declares interface dependencies */
2 typedef struct star_bus_manager {

```

```

3     star_bus_config_t*      buses;
4     SemaphoreHandle_t      mutex;
5     const char*            tag;
6     star_error_interface_t* error_iface; /* Injected */
7     star_pin_interface_t*   pin_iface;   /* Injected */
8     /* ... */
9 } star_bus_manager_t;
10
11 /* Initialization receives dependencies */
12 esp_err_t star_bus_manager_init(star_bus_manager_t* manager,
13                                 const char*          tag,
14                                 star_error_interface_t* error_iface,
15                                 star_pin_interface_t*   pin_iface)
16 {
17     ESP_RETURN_ON_FALSE(manager, ESP_ERR_INVALID_ARG, s_TAG,
18                         "Manager pointer is NULL");
19
20     manager->buses      = NULL;
21     manager->error_iface = error_iface; /* Can be NULL */
22     manager->pin_iface  = pin_iface;    /* Can be NULL */
23     /* ... */
24 }
25
26 /* Usage with injected dependencies */
27 error_handler_t error_handler;
28 error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
29
30 star_error_interface_t error_iface;
31 star_pin_interface_t pin_iface;
32 error_handler_get_interface(&error_iface, &error_handler);
33 pin_validator_get_interface(&pin_iface);
34
35 star_bus_manager_t bus_manager;
36 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

```

22.1.4 NULL Interface Handling

Components gracefully handle NULL interfaces.

```

1  /* Check interface before use */
2  if (manager->error_iface && manager->error_iface->record_error) {
3      manager->error_iface->record_error(manager->error_iface->ctx,
4                                          ret, "Operation failed",
5                                          __FILE__, __LINE__, __func__);
6  }
7
8  /* Or use convenience macro */
9  #define STAR_IFACE_RECORD_ERROR(iface, err, msg) \
10      ((iface) && (iface)->record_error \
11          ? (iface)->record_error((iface)->ctx, (err), (msg), \
12                                  __FILE__, __LINE__, __func__) \
13          : (err))

```

22.2 Bus Abstraction Pattern

The bus abstraction provides a unified interface for different communication protocols.

22.2.1 Unified Configuration

```

1  /* Single config type with protocol union */
2  typedef struct star_bus_config {
3      const char*      name;
4      star_bus_type_t type; /* Discriminator */
5      bool             initialized;
6      void*            handle;
7      void*            user_ctx;
8
9      union {
10         star_i2c_bus_config_t i2c;
11         star_spi_bus_config_t spi;
12         star_gpio_bus_config_t gpio;
13         star_adc_bus_config_t adc;
14     } proto;
15
16     struct star_bus_config* next;
17 } star_bus_config_t;

```

22.2.2 Factory Functions

Factory functions create configurations for specific protocols.

```
1  /* I2C bus factory */
2  star_bus_config_t* star_bus_config_create_i2c(
3      const char* name,
4      i2c_port_t  port,
5      uint8_t     address,
6      gpio_num_t  sda_pin,
7      gpio_num_t  scl_pin,
8      uint32_t    clk_speed);
9
10 /* SPI device factory */
11 star_bus_config_t* star_bus_config_create_spi_device(
12     const char*          name,
13     spi_host_device_t    host,
14     gpio_num_t           copi_pin,
15     gpio_num_t           cipo_pin,
16     gpio_num_t           sclk_pin,
17     int32_t              dma_chan,
18     const spi_device_interface_config_t* dev_cfg);
19
20 /* GPIO bus factory */
21 star_bus_config_t* star_bus_config_create_gpio(
22     const char* name,
23     gpio_num_t* pins,
24     uint8_t     pin_count);
```

22.2.3 Manager Pattern

The bus manager coordinates multiple bus configurations.

```
1  /* Initialize manager */
2  star_bus_manager_t bus_manager;
3  star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
4
5  /* Add buses */
6  star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
7      "sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22, 400000);
```

```

8 star_bus_manager_add_bus(&bus_manager, i2c_bus);
9
10 /* Find and use buses */
11 star_bus_config_t* bus = star_bus_manager_find_bus(&bus_manager, "sensor_i2c");
12
13 /* Safe access with callback (prevents use-after-free) */
14 esp_err_t my_callback(star_bus_config_t* bus, void* ctx) {
15     /* Access bus safely while mutex is held */
16     return ESP_OK;
17 }
18 star_bus_manager_with_bus(&bus_manager, "sensor_i2c", my_callback, NULL);

```

22.3 Operations Structure Pattern

Group related function pointers for pluggable implementations.

```

1 /* Define operations structure */
2 typedef struct star_i2c_ops {
3     star_i2c_write_fn_t      write;
4     star_i2c_read_fn_t      read;
5     star_i2c_write_command_fn_t write_command;
6     star_i2c_read_raw_fn_t   read_raw;
7 } star_i2c_ops_t;
8
9 /* Initialize with default implementations */
10 void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
11 {
12     if (ops == NULL) {
13         ESP_LOGE(s_TAG, "Cannot initialize NULL ops pointer");
14         return;
15     }
16     ops->write      = internal_star_bus_i2c_write;
17     ops->read       = internal_star_bus_i2c_read;
18     ops->write_command = internal_star_bus_i2c_write_command;
19     ops->read_raw    = internal_star_bus_i2c_read_raw;
20 }
21
22 /* Use through function pointers */
23 esp_err_t star_bus_i2c_write(const star_bus_manager_t* manager,

```

```

24         const char* name, ...)
25     {
26         star_bus_config_t* bus_config = star_bus_manager_find_bus(manager, name);
27         /* ... validation ... */
28         return bus_config->proto.i2c.ops.write(bus_config, data, len,
29                                             reg_addr, bytes_written);
30     }

```

22.4 Thread-Safe Callback Pattern

Safely access shared resources through callbacks.

```

1  /* Callback type */
2  typedef esp_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
3                                           void* user_ctx);
4
5  /* Thread-safe access function */
6  esp_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,
7                                     const char*      name,
8                                     star_bus_callback_t callback,
9                                     void*            user_ctx)
10 {
11     ESP_RETURN_ON_FALSE(manager && name && callback,
12                         ESP_ERR_INVALID_ARG, s_TAG, "Invalid arguments");
13
14     /* Acquire mutex */
15     if (xSemaphoreTake(manager->mutex,
16                         pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
17         != pdTRUE) {
18         return ESP_ERR_TIMEOUT;
19     }
20
21     /* Find bus while holding mutex */
22     star_bus_config_t* found_bus = NULL;
23     for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
24         if (curr->name && strcmp(curr->name, name) == 0) {
25             found_bus = curr;
26             break;
27         }

```

```

28     }
29
30     if (found_bus == NULL) {
31         xSemaphoreGive(manager->mutex);
32         return ESP_ERR_NOT_FOUND;
33     }
34
35     /* Invoke callback while holding mutex - prevents use-after-free */
36     esp_err_t callback_result = callback(found_bus, user_ctx);
37
38     xSemaphoreGive(manager->mutex);
39     return callback_result;
40 }

```

22.5 Thread Safety Patterns

22.5.1 Mutex Timeout Convention

Standard mutex timeout is 1000ms (configurable via Kconfig).

```

1  /* Standard mutex acquisition pattern */
2  if (xSemaphoreTake(manager->mutex,
3      pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
4      != pdTRUE) {
5      ESP_LOGE(manager->tag, "Timeout acquiring mutex");
6      return ESP_ERR_TIMEOUT;
7  }
8
9  /* Critical section */
10 /* ... protected operations ... */
11
12 xSemaphoreGive(manager->mutex);

```

22.5.2 Mutex with Cleanup (goto pattern)

```

1  esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2      star_bus_config_t* config)
3  {

```

```

4     ESP_RETURN_ON_FALSE(manager && config, ESP_ERR_INVALID_ARG,
5                          s_TAG, "Invalid arguments");
6
7     esp_err_t ret = ESP_OK;
8
9     if (xSemaphoreTake(manager->mutex,
10                        pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
11        != pdTRUE) {
12         return ESP_ERR_TIMEOUT;
13     }
14
15     /* Check for duplicate name */
16     for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
17         if (curr->name && strcmp(curr->name, config->name) == 0) {
18             ret = ESP_ERR_INVALID_STATE;
19             goto add_give_mutex; /* Single exit point */
20         }
21     }
22
23     /* Initialize bus hardware */
24     ret = star_bus_config_init(config, manager);
25     if (ret != ESP_OK) {
26         goto add_give_mutex;
27     }
28
29     /* Add to list */
30     config->next = manager->buses;
31     manager->buses = config;
32
33 add_give_mutex:
34     xSemaphoreGive(manager->mutex);
35     return ret;
36 }

```

22.6 Error Handler Ownership Pattern

Components track whether they own their error handler for proper cleanup.

```

1 /* Sensor handle with ownership tracking */

```

```

2 typedef struct mpu6050_handle {
3     star_bus_manager_t*    bus_manager;
4     const char*            bus_name;
5     star_error_interface_t* error_iface;
6     bool                   owns_error_handler; /* Ownership flag */
7     SemaphoreHandle_t      mutex;
8     bool                   initialized;
9     /* ... */
10 } mpu6050_handle_t;
11
12 /* Initialization with optional interface */
13 esp_err_t star_sensor_mpu6050_init(mpu6050_handle_t* handle,
14                                     star_bus_manager_t* bus_manager,
15                                     const char* bus_name,
16                                     star_error_interface_t* error_iface,
17                                     const mpu6050_config_t* config)
18 {
19     if (error_iface != NULL) {
20         /* Use provided interface - we don't own it */
21         handle->error_iface = error_iface;
22         handle->owns_error_handler = false;
23     } else {
24         /* Create default - we own it */
25         handle->error_iface = star_error_interface_create_default();
26         handle->owns_error_handler = true;
27     }
28     /* ... */
29 }
30
31 /* Cleanup respects ownership */
32 esp_err_t star_sensor_mpu6050_deinit(mpu6050_handle_t* handle)
33 {
34     /* Clean up error interface if we own it */
35     star_error_interface_cleanup(handle->error_iface,
36                                 handle->owns_error_handler);
37     handle->error_iface = NULL;
38     /* ... */
39 }

```

```

40
41 /* Helper for cleanup */
42 void star_error_interface_cleanup(star_error_interface_t* error_iface,
43                                 bool owns_handler)
44 {
45     if (!owns_handler || !error_iface) {
46         return; /* Not our responsibility */
47     }
48     error_handler_t* handler = (error_handler_t*)error_iface->ctx;
49     error_handler_destroy_default(handler);
50     free(error_iface);
51 }

```

22.7 Linked List Management

Bus configurations are managed as a singly-linked list.

```

1 /* Add to head of list (O(1)) */
2 config->next = manager->buses;
3 manager->buses = config;
4
5 /* Remove from list (O(n)) */
6 star_bus_config_t* prev = NULL;
7 for (star_bus_config_t* curr = manager->buses; curr; prev = curr, curr = curr->next
8     ) {
9     if (strcmp(curr->name, name) == 0) {
10         if (prev == NULL) {
11             manager->buses = curr->next; /* Remove head */
12         } else {
13             prev->next = curr->next; /* Remove middle/tail */
14         }
15         curr->next = NULL;
16         /* ... cleanup curr ... */
17         break;
18     }
19 }
20
21 /* Iterate through list */
22 for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {

```

```
22     /* Process each configuration */  
23 }
```

22.8 Best Practices Summary

- **Use dependency injection:** Components receive interfaces, not concrete types.
- **Handle NULL interfaces:** Always check before dereferencing.
- **Track ownership:** Know who is responsible for cleanup.
- **Use callbacks for safe access:** Prevents use-after-free with shared resources.
- **Consistent mutex timeout:** 1000ms standard timeout.
- **Single exit point with goto:** Ensures mutex release on all paths.
- **Factory functions:** Create configurations with proper initialization.
- **Operations structures:** Enable pluggable implementations.